

“Student Course Enrollment and Distributed Academic Management System”

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

**In DEPARTMENT OF COMPUTER SCIENCE ENGINEERING & DEPARTMENT
OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

By

V. ABHIDEV	2520030325
TEJ PRATAP SINGH	2520030476
G. SHASHANK	2520030495

Under the Esteemed Guidance of

Dr Yerragudipadu Subbarayudu Assistant Professor Department of Computer
Science and Engineering

Koneru Lakshmaiah Education Foundation (Deemed to be University estd. u/s. 3 of
the UGC Act, 1956) Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad,
Telangana - 500 043. Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE
ENGINEERING & DEPARTMENT OF COMPUTER SCIENCE AND
INFORMATION TECHNOLOGY **Declaration****

The Project Report entitled “Student Course Enrollment and Distributed Academic Management System” is a record of Bonafide work of V. ABHIDEV - 2520030325, TEJ PRATAP SINGH - 2520030476, G. SHASHANK - 2520030495 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

The investigations carried out in this project are original and represent genuine system development, database modeling, and empirical verification undertaken by the project team under supervision. Proper citations and references have been provided for all external literature, specifications, and libraries utilized during the development process.

**V. ABHIDEV – 2520030325 TEJ PRATAP SINGH – 2520030476 G. SHASHANK –
2520030495**

**K L (Deemed to be) University DEPARTMENT OF COMPUTER SCIENCE
ENGINEERING & DEPARTMENT OF COMPUTER SCIENCE AND
INFORMATION TECHNOLOGY **CERTIFICATE****

This is to certify that the mini project based report entitled “Student Course Enrollment and Distributed Academic Management System” is a bonafide work done and submitted by V. ABHIDEV - 2520030325, TEJ PRATAP SINGH - 2520030476, G. SHASHANK - 2520030495 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2024-2025.

The design, implementation, and database architecture presented in this report have been verified through active laboratory demonstrations and fulfill the academic rigor required for Database Software Engineering and Distributed Backend Development.

Signature of the Guide

Signature of the Course Coordinator	Signature of the HOD
--	-----------------------------

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our Course Coordinator Dr Yerragudipadu Subbarayudu, and our guide Dr Yerragudipadu Subbarayudu, Department of Computer Science and Engineering, which cannot be expressed in words for his tremendous support, encouragement, technical insights, and continuous guidance throughout the duration of this project.

We express our deep gratitude to Dr. P Venkateshwara Rao (CSE) / Dr. Kaja Shareef (CSIT), Head of the Department for Computer Science Engineering / Department of Computer Science and Information Technology, for providing us with adequate laboratory facilities, computing infrastructure, and ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members who have assisted us directly or indirectly for the successful completion of this project. Finally, we sincerely thank our parents, friends, and classmates for their kind help, moral support, and cooperation during our work.

**V. ABHIDEV – 2520030325 TEJ PRATAP SINGH – 2520030476 G. SHASHANK –
2520030495**

Abstract

The “Student Course Enrollment and Distributed Academic Management System” is a web-based academic portal designed to organize student information, course offerings, enrollments, grades, timetable details, notifications, and administrative records. The system provides separate user-facing pages for common student activities and administrative pages for reviewing academic records. Its purpose is to reduce fragmentation in academic workflows and demonstrate practical database and full-stack development concepts.

The frontend is implemented with React and React Router, while the backend uses Node.js and Express to expose REST-style API endpoints. The frontend communicates with the server through a centralized API service that sends and receives JSON. The backend uses a MySQL connection pool through mysql2 and loads database settings from environment variables. The submitted source includes route modules for authentication, students, courses, enrollments, grades, timetable, notifications, and administration.

The MySQL schema, named `student_portal_db`, defines relational tables and constraints for academic entities. It uses primary keys, foreign keys, unique constraints, CHECK constraints, indexes, analytical views, a stored procedure, a stored function, and enrollment audit triggers. Enrollment and course dropping are executed as transactions so that related changes are applied consistently, and database behavior is verified through the test cases documented in Chapter 8.

The project demonstrates the integration of a browser-based interface, a server-side API, and a relational database. It provides a practical case study in React component organization, Express routing, SQL schema design, data integrity, API integration, and test planning. The documentation distinguishes implemented functionality from proposed enhancements (Chapter 11) and records the test cases and their outcomes in Chapter 8.

From a data architecture perspective, the platform employs MySQL as its relational database management system to preserve and organize vital institutional records. Through the strategic implementation of normalized tables, primary and foreign key constraints, and relational logic, the schema guarantees high data integrity while minimizing redundancy. Dynamic SQL queries facilitate real-time data manipulation, including insertion, modifications, deletion, and extraction of academic inventories.

The application’s backend is constructed around RESTful APIs, bridging client-side requests with core database operations. The server layer handles request routing, input validation, and JWT-based authentication to safeguard sensitive administrative records. Furthermore, the

architecture is modular, with separate route modules for student administration, course cataloging, and enrollment processing, and it supports academic cluster cohorts that group course sections.

Index

S.No.	Chapters	Topics	Page Range
	Front Matter	Declaration, Certificate, Acknowledgement, Abstract	1 - 5
1	Introduction	1.1 Background to 1.8 Report Organization	8 - 11
2	System Requirements	2.1 Hardware, Software, Functional, Non-Functional	12 - 15
3	Technology Stack	React, React Router, Node.js, Express, MySQL, REST APIs	16 - 19
4	System Architecture	Layered Architecture, Request Sequence, Deployment	20 - 23
5	Design	Relational Model, ER Relationships, Schema, Views, Procedures	24 - 37
6	Implementation	Frontend Pages, API Integration, Auth, Feature Modules	38 - 43
7	Features	Login, Catalog, Enrollment, Timetable, Grades, Admin	44 - 49
8	Testing	Testing Strategy, Test Cases TC-01 to TC-14, Evidence	50 - 54
9	Deployment	Local Setup, MySQL Config, Security Checklist, Recovery	55 - 57
10	Challenges & Limitations	Frontend-Backend Sync, Schema Compatibility, Concurrency	58 - 59
11	Future Enhancements	Authentication, Analytics, Search, Timetable, Scaling	60 - 61

12	Conclusion	Project Summary, Outcomes, Skills Learned, Final Remarks	62 - 63
13	References	Source Files, Documentation, Standards, Citations	64 - 65
14	Appendices	Directory Tree, API Reference, SQL Examples, User Guides	66 - 84

CHAPTER 1 – INTRODUCTION

This chapter establishes the purpose, institutional context, and boundaries of the Student Course Enrollment and Distributed Academic Management System. It identifies the critical academic tasks addressed by the submitted implementation, defines core objectives, and outlines the structural organization of this project report.

1.1 Background of the Project

Academic institutions maintain voluminous, mission-critical information about students, curricula, course offerings, faculty allocations, grades, timetables, and dynamic enrollment activity. When these records are managed through disconnected spreadsheets, localized flat-files, or manual administrative procedures, the same information is frequently recorded multiple times. This introduces severe data redundancy, inconsistent state updates, and administrative overhead where stakeholders find it difficult to identify authoritative data records.

The Student Course Enrollment and Distributed Academic Management System provides an integrated, web-based digital portal for accessing and orchestrating these related academic services through a standardized, consistent workflow. Designed specifically to solve real-world university management challenges, the platform centralizes student registration, course offerings, capacity-controlled course registrations, and transcript generations into a unified relational database schema.

The submitted implementation utilizes a modern, reactive React frontend, an asynchronous Node.js and Express RESTful API layer, and an enterprise MySQL 8.0 relational database engine for persistent transactional storage. The active schema is designated as `student_portal_db`. The software architecture establishes distinct student self-service pages alongside comprehensive administrative oversight consoles, supported by secure server-side route handlers that enforce strict academic constraints.

1.2 Problem Statement

The central challenge addressed by this project is the critical requirement to coordinate multi-entity academic records and course enrollment operations while maintaining absolute data consistency across distributed client sessions. Students require instantaneous visibility into available course catalogs, real-time section capacities, current enrollment statuses, and

historical grade sheets. Simultaneously, academic administrators must oversee course inventories, allocate faculty instructors, configure student quotas, and monitor enrollment metrics without experiencing data corruption.

Conventional academic software solutions often suffer from severe synchronization latencies, orphan foreign key records, race conditions during high-volume registration periods, and lack of cohort synchronization. When user interfaces and database tiers are decoupled without robust transactional boundaries, displayed seat counts become stale, leading to unintentional over-enrollments and schedule collisions.

The project resolves these persistent vulnerabilities through a centralized 3NF relational schema, parameterized REST API endpoints, and a responsive frontend client. Strict database-level domain and referential integrity constraints prevent duplicate student identifiers and invalid course mappings. Atomic database transactions and triggers safeguard transactional consistency during high-concurrency enrollment and course-drop actions.

1.3 Scope of the Project

The functional scope of the implemented system covers authenticated student registration and sign-in workflows, student academic profile retrieval and updates, interactive course catalog exploration, real-time capacity-checked course enrollment, verified course-drop workflows, letter grade and GPA computation views, automated timetable scheduling, notification dispatching, and administrative oversight consoles.

Within the administrative domain, authorized personnel are provided with consolidated telemetry dashboards tracking institutional student counts, course occupancy percentages, faculty teaching assignments, and student enrollment audit trails. Advanced features include academic cluster cohorts, which group course sections and enforce semester-wide cohort enrollment locks to preserve academic curriculum synchronization.

The scope is strictly bound to the verified capabilities implemented within the submitted source codebase. It deliberately excludes integrations with external third-party payment gateways, commercial single-sign-on (SSO) OAuth2 federation with external identity providers, or cloud-hosted microservices infrastructure. The platform operates as a cohesive, modular, on-premises full-stack software system designed for university campus deployment.

1.4 Objectives

The primary technical objective of this project is to architect, develop, and empirically validate an integrated academic management system that demonstrates core database engineering and distributed backend development principles. The system aims to replace fragmented legacy record-keeping with an ACID-compliant relational framework capable of serving concurrent student and administrative workflows.

Specific operational objectives include: (1) Designing a normalized MySQL relational schema enforcing primary, foreign, candidate, and domain CHECK constraints; (2) Engineering a secure Node.js and Express backend providing stateless RESTful API endpoints; (3) Implementing atomic transactions and automated triggers to handle course enrollments and audit logging; (4) Developing an interactive React client interface delivering seamless user experiences; (5) Formulating analytical views and window functions for automated academic performance metrics.

A further pedagogical objective is the practical application of enterprise database concepts: index optimization on high-cardinality keys, multi-table JOIN query planning, parameterized SQL execution for SQL injection mitigation, and automated transaction rollback mechanisms during unexpected server or database failures.

1.5 Importance of the Application

Modern educational institutions require dependable, fault-tolerant digital infrastructure to administer academic programs. By establishing an integrated database management system, universities eliminate human data-entry errors, ensure regulatory compliance, provide transparent auditability for all registration events, and democratize access to academic standing metrics for enrolled students.

For students, the application provides self-service empowerment: learners can independently browse course descriptions, inspect prerequisite requirements, register for desired cohorts, review individualized weekly timetables, and monitor cumulative grade point averages in real time without navigating administrative queues.

From an engineering and educational standpoint, this project bridges theoretical database design principles with full-stack software engineering. It illustrates the complete end-to-end lifecycle of academic data: from client-side state collection, through HTTP request marshaling, middleware token validation, and connection pool allocation, to binary database execution on disk.

1.6 Target Users and Audience

The primary user groups interacting with the portal are categorized into two major operational roles: Registered Students and Academic Administrators. Students utilize the self-service interface to register their accounts, authenticate their sessions, browse course offerings, enroll in courses within their assigned academic cluster, track their class schedules, and review published examination grades.

Academic Administrators represent department chairs, registrars, and academic evaluators. Administrators utilize administrative dashboards to monitor campus-wide enrollment occupancy, inspect student demographics, manage academic clusters, enforce CGPA overrides when sanctioned by academic committees, and audit historical course drop/add events for compliance.

Faculty data is systematically modeled within the database schema to represent teaching staff qualifications, departments, and course assignments. Although faculty records are managed through administrative views in the current release, the modular architecture lays the structural groundwork for future dedicated faculty portals.

1.7 Project Overview

The system implements a classic 3-Tier Layered Architecture consisting of a Presentation Tier (React SPA), an Application Logic Tier (Express REST API), and a Data Persistence Tier (MySQL InnoDB RDBMS). Client browsers execute compiled React components that initiate asynchronous fetch calls using standardized JSON envelopes to the backend service.

The Node.js server receives inbound HTTP requests, applies security and CORS headers, parses JSON request bodies, and executes route-specific controllers. Database interactions are executed over a persistent connection pool using the mysql2/promise driver, ensuring high throughput, parameter binding against SQL injection, and automatic connection reuse.

The database tier enforces relational integrity at the storage level. All table creations adhere to third normal form (3NF), foreign key cascades are deliberately controlled, and transactional routines ensure that concurrent requests cannot violate capacity constraints or corrupt student enrollment records.

1.8 Report Organization

This documentation report is structured into 14 distinct, logically sequential chapters that systematically examine the software system from conception to empirical validation. Chapter 1 introduces the project background, problem statement, scope, and objectives. Chapter 2 details the hardware, software, functional, and non-functional system requirements. Chapter 3 examines the complete technology stack.

Chapter 4 presents the high-level system architecture, component topologies, and request sequence traces. Chapter 5 explores the database design, entity relationships, normalization proofs, views, triggers, and stored routines. Chapter 6 details the implementation of frontend and backend modules. Chapter 7 covers user-facing and administrative features.

Chapter 8 formulates the comprehensive testing strategy, test case matrix, and empirical evidence. Chapter 9 outlines deployment and configuration procedures. Chapter 10 analyzes development challenges and current limitations. Chapter 11 proposes future technical enhancements. Chapter 12 delivers the project conclusion. Chapter 13 provides academic references, and Chapter 14 supplies exhaustive technical appendices.

CHAPTER 2 – SYSTEM REQUIREMENTS

This chapter outlines the minimum and recommended hardware configurations, software dependencies, operating system runtimes, development toolsets, functional specifications, and non-functional quality standards necessary to construct, deploy, and maintain the academic portal.

2.1 Hardware Requirements for Development

The development environment requires workstation hardware capable of executing multiple concurrent background services: the Node.js runtime process, the Vite development server with hot module replacement, the MySQL 8.0 server daemon, an integrated development environment (IDE), and browser developer inspection tools.

Minimum hardware specifications comprise a dual-core 64-bit processor (Intel Core i3 or AMD Ryzen 3 equivalent, 2.0 GHz or higher), 8 GB of physical RAM, and 10 GB of available solid-state storage. This baseline ensures that database connection pools and development build caches operate without thrashing virtual memory swap files.

Recommended workstation specifications comprise a quad-core or hexa-core processor (Intel Core i5/i7 or AMD Ryzen 5/7), 16 GB of DDR4/DDR5 RAM, and high-speed NVMe SSD storage. Additional memory capacity substantially accelerates Vite compilation cycles and enables smooth operation of MySQL Workbench alongside active client sessions.

2.2 End-User Hardware Requirements

End-user client hardware requirements are designed to be lightweight and accessible, as all computational processing, query evaluation, and business logic execution occur on the server and database tiers. End users interact with the system strictly via standard web browsers.

Client devices may include standard desktop computers, laptops, chromebooks, tablets, or modern smartphones. Hardware requirements for the client device are limited to an active display resolution of 1024x768 pixels or higher (with responsive support down to 360px mobile viewports), 2 GB of RAM, and a stable network connection capable of handling HTTP/1.1 or HTTP/2 TCP traffic.

Because client browsers receive pre-compiled JavaScript, HTML, and CSS bundles from the web server, no local database drivers, compilers, or server utilities need to be installed on the client machine, guaranteeing seamless universal accessibility across diverse campus devices.

2.3 Operating System and Runtime

The application backend and development toolchains are built upon cross-platform web standards, enabling seamless operation across Microsoft Windows (Windows 10, Windows 11, Windows Server), POSIX Linux distributions (Ubuntu 20.04+, Debian, Fedora, CentOS), and Apple macOS.

The primary runtime environment is Node.js, requiring version 20.0.0 LTS or higher (the current verified production environment utilizes Node.js v24.14.1). Node.js provides the asynchronous, non-blocking V8 JavaScript engine that executes Express route handlers and manages database socket connections.

Package management is orchestrated through npm (v10+ or v11+), which resolves project dependencies, validates package checksums via lockfiles, and executes lifecycle automation scripts for compiling, testing, and starting server daemons.

2.4 Software Requirements

The core database server requirement is MySQL Community Server version 8.0.16 or higher (specifically configured with the InnoDB storage engine and utf8mb4 collation). MySQL manages relational schema tables, primary and foreign key constraints, analytical views, and stored procedures.

Client-side software requirements entail any evergreen web browser with full ECMAScript 2020+ and CSS Flexbox/Grid support, including Google Chrome (version 100+), Mozilla Firefox (version 100+), Microsoft Edge, or Apple Safari.

Administrative database management is facilitated by MySQL Workbench 8.0 CE or equivalent GUI clients (such as DBeaver or Navicat). API verification, contract testing, and endpoint simulation are supported by Postman (v10+) or standard curl command-line utilities.

2.5 Development Tools and Frameworks

The application framework stack incorporates React 18 for building declarative user interfaces, React Router DOM v6 for client-side single-page routing, and Vite 5 as the next-generation frontend build tooling and development server.

The backend server is constructed using Express.js 4.19, complemented by essential middleware packages: cors for cross-origin resource sharing, dotenv for environmental secret isolation, and mysql2 for high-throughput promise-based database connectivity.

Source code authoring and debugging are conducted in Visual Studio Code, augmented with extensions for JavaScript (ES6+), React JSX, SQL syntax highlighting, and Git version control integration. The esbuild bundler provides lightning-fast production asset compilation.

2.6 Functional Requirements

The functional requirements specify the operational capabilities delivered by the software system: (1) FR-01: Secure student and administrative authentication with password verification and stateless JWT token issuance; (2) FR-02: Self-service student registration with unique roll number and institutional email validation; (3) FR-03: Real-time course catalog exploration displaying course code, title, credits, schedule, classroom, and live seat availability.

(4) FR-04: Academic cluster selection allowing students to select a cohort track that locks section offerings; (5) FR-05: Atomic course enrollment validating seat capacity, cluster membership, and duplicate enrollment constraints; (6) FR-06: Verified course-drop functionality updating enrollment status and releasing reserved seat capacity; (7) FR-07: Automated timetable generation compiling active enrollments into structured weekly class schedules.

(8) FR-08: Academic transcript and grade viewing presenting internal, midterm, and final assessment scores; (9) FR-09: Student notification center tracking enrollment confirmations and administrative alerts; (10) FR-10: Administrative telemetry console displaying campus-wide enrollment statistics, occupancy percentages, and audit logs.

2.7 Non-Functional Requirements

The system adheres to rigorous non-functional quality standards: (1) Performance: API endpoint response latencies must remain below 100 milliseconds for standard read operations under nominal local loads; database queries must leverage secondary B-Tree indexes to prevent full table scans.

(2) **Data Integrity:** The database must strictly enforce ACID transactional properties; foreign key constraints must prohibit orphan child records, and domain CHECK constraints must ensure that marks (0–100), credits (1–6), and GPAs (0.0–10.0) remain within valid ranges.

(3) **Reliability and Availability:** The system must handle unexpected client disconnection or invalid payloads gracefully without crashing the Node.js process; database connection pools must automatically reconnect after transient network dropouts; (4) **Security:** Passwords must be hashed using cryptographic algorithms; all API parameters must use prepared statements to prevent SQL injection vulnerabilities.

2.8 Assumptions and Constraints

System development and documentation are formulated under specific engineering assumptions: (1) The application is designed to operate against a local or private intranet MySQL 8.0+ server instance; (2) Relational SQL scripts employ MySQL-specific procedural syntax (DELIMITER, stored procedures, DECLARE) and are not directly portable to PostgreSQL or Oracle without dialect translation.

(3) Academic terms are structured on a trimester/semester basis where course enrollments belong to the active academic term; (4) Sample student profiles, faculty entries, and assessment grades provided in the seed files represent realistic development datasets and do not expose confidential institutional records.

(5) Network connectivity between the Express API server and MySQL daemon is assumed to have minimal latency (<5ms) within the local host loopback interface (127.0.0.1:3306).

CHAPTER 3 – TECHNOLOGY STACK

This chapter presents an in-depth analysis of the technologies, architectural libraries, runtimes, and database engines comprising the Student Course Enrollment and Distributed Academic Management System stack.

3.1 Front-End Overview

The client-side presentation layer is implemented as a modern Single Page Application (SPA) powered by React 18. React's component-based model decomposes complex academic views—such as course catalogs, enrollment planners, and grade transcripts—into modular, reusable, and testable UI elements.

By maintaining an in-memory Virtual DOM representation, React efficiently computes minimal layout diffs and batches DOM mutations, ensuring fluid, responsive client interactions without full page reloads. This is vital during high-frequency enrollment operations where immediate visual feedback is essential.

The application utilizes modern React design paradigms, including functional components, custom hooks, and the Context API for shared global state. The compilation pipeline is driven by Vite 5, providing rapid Hot Module Replacement (HMR) and optimized production minification.

3.2 HTML, CSS, and Browser Rendering

The user interface styling is authored in clean, modular Vanilla CSS without reliance on heavy utility frameworks or bloated component libraries. This approach provides fine-grained control over layout rendering, prevents style pollution, and ensures high performance.

Stylesheets are partitioned logically into four dedicated modules: `global.css` (design tokens, CSS variables, typography, and reset rules), `layout.css` (grid containers, responsive flex wrappers, headers, and navigation sidebars), `components.css` (buttons, badges, cards, modals, and form controls), and `pages.css` (page-specific layout overrides).

Modern CSS features such as CSS custom properties (variables for harmonious academic color palettes), Flexbox, CSS Grid, and media queries guarantee responsive adaptation across standard desktop displays, tablets, and mobile smartphones while preserving WCAG accessibility contrast ratios.

3.3 React and Component-Based Design

The frontend follows atomic design principles. Reusable presentation components reside in `src/components/`, including Header (user identity, notifications, logout triggers), Sidebar (role-based navigation links), StatCard (metric visualization), CourseCard (course metadata and enrollment triggers), Modal (confirmation dialogues), and ToastContainer (ephemeral alerts).

Global application state is managed by `AppContext` located in `src/context/AppContext.jsx`. The context provider centralizes authentication tokens, active user profiles, course inventories, active enrollments, and notification state. This architecture eliminates deep prop drilling across intermediate components.

When a student performs an enrollment or drop action, context dispatcher functions trigger background API calls and synchronize local state upon successful server confirmation, ensuring that UI indicators and seat counts update immediately.

3.4 Routing and Protected Pages

Client-side navigation and URL synchronization are orchestrated by React Router DOM version 6 in `src/AppRoutes.jsx`. The route tree defines explicit mappings between browser URLs and high-level page views.

To safeguard sensitive academic resources, the architecture implements a custom `ProtectedRoute` component wrapper. Unauthenticated sessions attempting to access protected routes (such as `/dashboard`, `/courses`, `/grades`, or `/admin`) are intercepted and redirected to the login view with state history preserved.

Furthermore, the routing system enforces role-based access control (RBAC). Dedicated administrative routes (such as `/admin`, `/admin/students`, `/admin/clusters`) verify the authenticated user's role and block standard student accounts from accessing privileged administrative screens.

3.5 Back-End Overview

The application backend is built upon Node.js and Express.js (v4.19). Node.js provides a single-threaded, event-driven runtime utilizing libuv to execute non-blocking, asynchronous I/O operations, making it ideally suited for I/O-intensive database web applications.

The Express server initializes in `server/server.js` and mounts its primary application logic in `server/app.js`. The server applies essential security middleware, configures JSON body parsers with payload size limits (64kb), and mounts decoupled route modules.

Endpoints are organized modularly: `auth.routes.js` (login, registration), `students.routes.js` (profile, directory), `courses.routes.js` (catalog, prerequisites), `enrollments.routes.js` (registration, dropping), `grades.routes.js` (transcripts), `timetable.routes.js` (schedules), `notifications.routes.js` (alerts), `clusters.routes.js` (cohorts), and `admin.routes.js` (telemetry).

3.6 MySQL and mysql2

Persistent data storage is powered by MySQL 8.0, utilizing the transactional InnoDB storage engine. MySQL provides row-level locking, foreign key enforcement, multi-version concurrency control (MVCC), and crash-safe write-ahead logging (WAL).

Node.js interacts with MySQL via the `mysql2/promise` driver configured in `server/config/db.js`. The driver creates an optimized connection pool with a maximum connection limit of 10, connection queuing, and automatic thread recycling, preventing resource exhaustion during traffic spikes.

All SQL queries execute via parameterized prepared statements (using `?` placeholders). This architectural separation of query structure and user-supplied data completely neutralizes SQL injection threats and optimizes query execution plan caching within the MySQL engine.

3.7 REST APIs and JSON

Client-server communication strictly adheres to REST (Representational State Transfer) architectural constraints. Communication occurs over HTTP using JSON (JavaScript Object Notation) as the universal serialization format.

Endpoints follow predictable RESTful noun naming conventions: `GET /api/courses` (retrieve catalog), `POST /api/enrollments` (create enrollment), `DELETE /api/enrollments` (drop enrollment), and `GET /api/grades/:studentId` (retrieve student transcript).

Responses are encapsulated within a standardized JSON response envelope containing success (boolean), message (string), and payload data objects. HTTP status codes communicate outcome semantics accurately: 200 OK for successful reads, 201 Created for additions, 400 Bad Request for validation errors, 401 Unauthorized for invalid tokens, 409 Conflict for duplicate keys, and 500 for server errors.

3.8 Environment Configuration

The backend adheres to Twelve-Factor App methodology by decoupling application logic from environment configuration. Sensitive connection parameters and secrets are loaded into `process.env` at boot time via `dotenv` from `server/.env`.

Configurable environment variables include `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD` (or `DB_PASSWORD_B64` for base64 encoded strings), `DB_NAME`, `PORT`, and `JWT_SECRET`. The server enforces a mandatory security check verifying that `JWT_SECRET` contains at least 32 cryptographically secure characters.

To prevent inadvertent credential leakage, `server/.env` is explicitly listed in `.gitignore`, while `server/.env.example` provides sanitized template directives for onboarding new development workstations.

3.9 Version Control and Build Tools

Project source code is maintained under Git version control, following standard branch isolation practices. The repository structure separates client source code (`src/`) from server route controllers (`server/`) and database initialization scripts (`server/db/`).

Frontend compilation is executed via `npm` scripts. Running `npm run build` triggers `scripts/build.mjs`, which leverages the blazing-fast `esbuild` compiler to bundle and minify JavaScript modules into a compact `dist/assets/app.js` artifact, accompanied by pre-rendered production HTML.

Development orchestration utilizes concurrently (`npm run dev`) to spawn both the Express server in watch mode (`node --watch`) and the Vite frontend dev server in parallel, providing seamless developer ergonomics.

3.10 Technology Stack Summary

The assembled technology stack—React 18, Node.js 24, Express 4.19, and MySQL 8.0—represents a mature, robust, and highly maintainable architecture for academic enterprise software. The list below summarizes the stack components and their respective architectural responsibilities.

Presentation Layer: React 18, React Router v6, Vanilla CSS (Dynamic UI rendering and client-side routing); Application Layer: Node.js, Express.js (REST API orchestration, token

authorization, request validation); Database Layer: MySQL 8.0, InnoDB Engine (Transactional persistence, ACID integrity, analytical views); Tooling: Vite 5, esbuild, npm, Git, Postman, MySQL Workbench (Compilation, testing, and administration).

This decoupled full-stack architecture achieves a high degree of maintainability, modular testability, and security, ensuring that database consistency rules are enforced at the storage engine level regardless of client behavior.

CHAPTER 4 – SYSTEM ARCHITECTURE

This chapter presents the architectural blueprints, structural layers, request sequence flows, and deployment topologies of the Student Course Enrollment and Distributed Academic Management System.

4.1 High-Level Architecture

The system is organized around a 3-Tier Layered Architecture that establishes clear separation of concerns across presentation, business logic, and data persistence. Each tier is decoupled through standardized interface boundaries, ensuring modularity, security, and independent maintainability.

The Presentation Tier executes entirely within the user's browser environment, providing interactive views and capturing user events. The Application Logic Tier runs within a Node.js process hosting an Express HTTP server; it acts as an authenticated gateway, validating client input, verifying session tokens, and orchestrating business workflows.

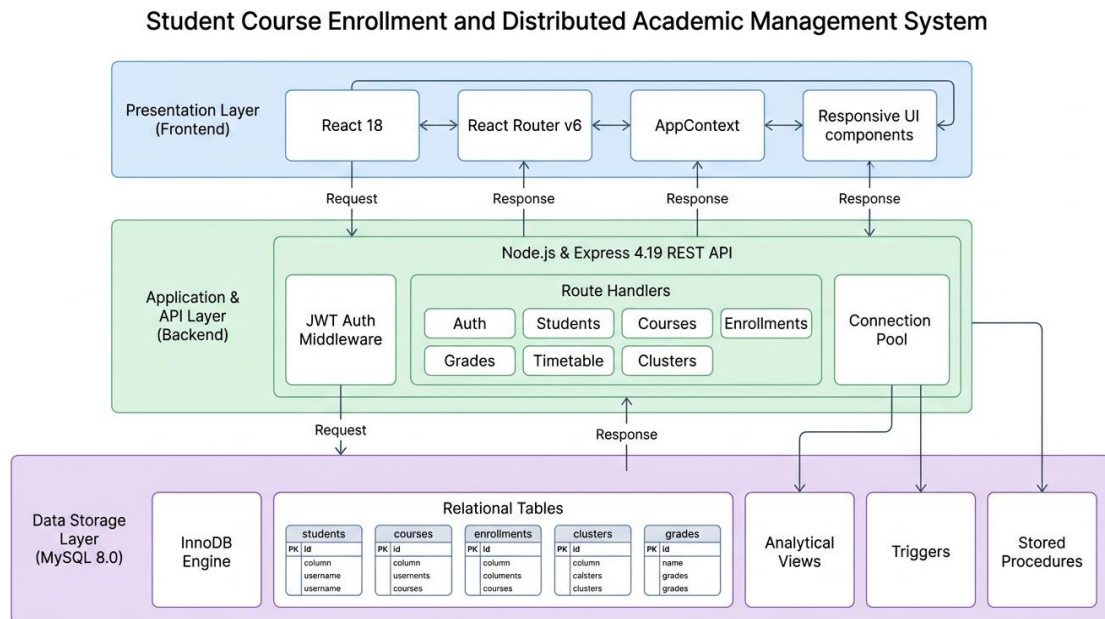
The Data Persistence Tier resides in the MySQL RDBMS, which manages persistent storage on disk, executes relational queries, evaluates triggers, and enforces referential integrity constraints across academic entities.

4.2 Architecture Diagram

The system architecture follows a clean horizontal tiered topology with well-defined communication channels. The diagram below illustrates the comprehensive request-response flow across the Presentation Layer, the Application/API Layer, and the MySQL Database Layer.

Client requests initiated from React components pass through a centralized API service and travel over HTTP to the Express backend. The server processes requests through authentication and validation middlewares before routing them to domain-specific controllers. Database queries are executed over pooled connections to MySQL 8.0.

Responses generated by the database engine return through the connection pool, where the Express server formats them into standardized JSON envelopes and returns them to the React client to update local UI state.



4.3 Presentation Layer

The Presentation Layer encompasses all visual elements, client routing, layout containers, and user interaction logic. Built as a Single Page Application using React 18, it renders dynamic views without triggering full page reloads, providing desktop-class application performance.

The layer contains three primary subsystems: (1) View Components: Dashboard, CourseCatalog, CourseDetails, MyCourses, Timetable, Grades, Notifications, StudentProfile, and AdminManagement; (2) Shared UI Library: Navbars, Sidebars, Modals, Badges, and Alert Toasts; (3) State Management: AppContext, which maintains active user identity, token caches, and synchronized academic data.

The Presentation Layer is strictly decoupled from direct database communication. It interacts with the backend solely through asynchronous HTTP fetch operations via `src/services/api.js`, ensuring that database credentials and server logic remain inaccessible to client browser inspection.

4.4 API and Application Layer

The Application Layer is hosted on a Node.js runtime and driven by Express.js 4.19. It exposes RESTful HTTP endpoints listening on port 5000 (configurable via environment variables) and coordinates incoming client requests with database operations.

Upon receiving an inbound request, the layer executes a structured middleware pipeline: (1) Security Headers: sets X-Content-Type-Options, X-Frame-Options, and Referrer-Policy; (2)

Body Parsing: decodes JSON payloads with a strict 64kb limit; (3) Authentication Filter: verifies Bearer JWT tokens in authorization headers; (4) Rate Limiting: restricts excessive authentication attempts to prevent brute-force attacks.

Validated requests are dispatched to dedicated controller routes that execute business logic, query the MySQL connection pool, and format JSON responses. A centralized error handling middleware catches unhandled exceptions, maps database error codes (such as ER_DUP_ENTRY and ER_NO_REFERENCED_ROW_2) to appropriate HTTP statuses, and logs diagnostic traces.

4.5 Data Layer

The Data Layer is implemented in MySQL 8.0, utilizing the InnoDB storage engine for all database tables. InnoDB provides ACID transaction compliance, row-level locking for high concurrency, foreign key constraints, and crash recovery via write-ahead logging.

The database schema (student_portal_db) organizes academic entities into normalized relational tables. It defines declarative constraints including PRIMARY KEY, UNIQUE, FOREIGN KEY, and domain CHECK constraints to guarantee data integrity at the storage layer.

Beyond static tables, the Data Layer incorporates advanced database programming constructs: analytical views (v_student_academic_summary, v_course_enrollment_stats, v_student_rankings), automated audit triggers (tr_enrollment_insert, tr_enrollment_update), a stored procedure (sp_student_transcript), and a stored function (fn_calculate_student_avg).

4.6 Enrollment Request Sequence

The course enrollment workflow represents the core transactional sequence of the system. It executes through the following coordinated steps: (1) User Action: A student selects a course section on the frontend and clicks 'Enroll Now'; (2) API Dispatch: src/services/api.js transmits a POST request to /api/enrollments with courseId and sectionId.

(3) Authentication Check: The authenticate middleware validates the student's JWT token, extracting the verified student ID; (4) Business Validation: The enrollment controller verifies that the student is active, checks academic cluster alignment, and ensures the student has not already enrolled in this course.

(5) Database Transaction: A connection is acquired from the pool; a query checks seat availability; an INSERT is executed on the enrollments table; (6) Trigger Execution: The MySQL trigger `tr_enrollment_insert` fires automatically, writing an immutable record to `enrollment_audit_log`; (7) Notification: A notification record is inserted; (8) Response: A 201 Created JSON response is returned, triggering a UI state refresh on the client.

4.7 Course-Drop Request Sequence

The course-drop sequence provides an atomic, reversible mechanism for withdrawing from an enrolled course: (1) User Action: The student navigates to 'My Courses' and initiates a drop request; (2) API Request: A DELETE request is transmitted to `/api/enrollments` with `studentId` and `courseId`.

(3) Verification: The server validates authentication and checks that an active enrollment record exists with status 'Enrolled'; (4) Status Transition: The backend executes an UPDATE setting `status = 'Dropped'`; (5) Audit Trigger: The MySQL trigger `tr_enrollment_update` fires immediately, recording a 'DROPPED' event in `enrollment_audit_log`.

(6) Capacity Adjustment: Available seats in the course are recalculated; (7) Timetable Synchronization: Corresponding timetable slots are removed; (8) Confirmation: The server returns 200 OK, and the frontend updates the student's enrolled courses and timetable views.

4.8 Deployment Architecture

The system is architected for both streamlined local development and robust on-premises campus deployment. In local development mode, Vite serves frontend modules on port 5173 while Express listens on port 5000, connected via proxy routing.

For production deployment, the frontend React application is pre-compiled and bundled into optimized static assets within the `dist/` directory. The Express backend serves these static production assets directly from its root path while exposing the API on `/api/*`, consolidating the entire full-stack application into a single unified port (5000).

Database connectivity is maintained over local loopback (127.0.0.1:3306), isolated from public network interfaces. A production checklist mandates SSL/TLS termination via a reverse proxy (such as Nginx), environment secret rotation, and automated daily MySQL `mysqldump` backup routines.

CHAPTER 5 – DESIGN

This chapter details the relational data model, entity-relationship structures, normalization proofs, schema constraints, indexing strategies, analytical views, and stored database routines defining the Student Course Enrollment database design.

5.1 Design Approach

The database design adopts a rigorous entity-centric relational modeling approach. Academic entities—including students, academic administrative staff, courses, faculty instructors, course sections, syllabi, enrollments, assessment grades, attendance logs, timetables, notifications, and academic clusters—are partitioned into dedicated relational tables to enforce strict data ownership and eliminate redundancy.

The schema utilizes string natural identifiers for primary academic entities (such as student roll numbers, course codes, and cluster identifiers) to maintain human-readable alignment with university registry standards. For high-frequency transactional and audit records (such as enrollments, grades, and audit trails), auto-increment surrogate integer keys are employed to optimize indexing performance.

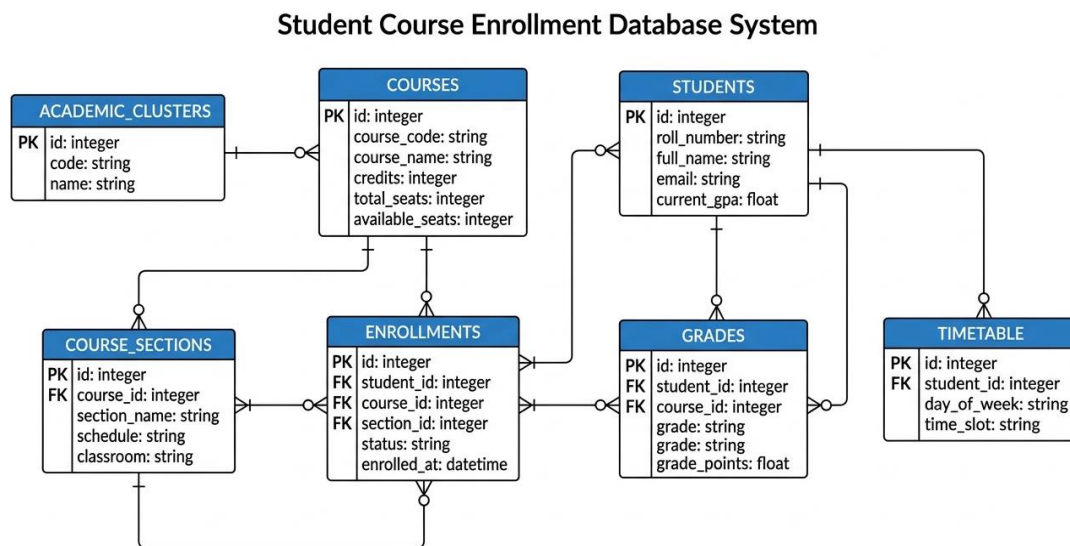
Referential integrity is guaranteed through comprehensive foreign key constraints with explicit ON DELETE / ON UPDATE cascade and restrict policies. Domain integrity is enforced at the database engine level through declarative CHECK constraints, ensuring that invalid academic data cannot enter the system.

5.2 ER Diagram and Relationships

The Entity-Relationship (ER) model centers upon the fundamental Many-to-Many (M:N) relationship between Students and Courses. Because a student may enroll in multiple courses and a course section accommodates multiple students, this relationship is resolved through the associative Enrollments entity, which records individual student-course-section associations with status and timestamps.

The diagram below presents the complete Entity Relationship Diagram modeled in Crow's Foot notation, detailing entity attributes, primary keys (PK), foreign keys (FK), and relationship cardinalities across all database entities including Students, Courses, Course Sections, Grades, Timetable, and Academic Clusters.

Courses maintain 1:N identifying relationships with Course Syllabus, Course Outcomes, and Course Assessments, and a 1:N relationship with Course Sections. Students maintain 1:N relationships with Grades, Attendance, Timetable slots, Notifications, and Audit entries. Academic Clusters group course sections through Cluster Groups, establishing cohesive cohort boundaries.



5.3 Students Table

The students table represents the authoritative repository for student identity, institutional affiliation, and cumulative academic metrics. Its primary key is id (VARCHAR(50)), with roll_number and email defined with UNIQUE constraints to prevent duplicate identity creation.

Attributes include full_name, password_hash, phone, department, program, year_level, semester, current_gpa, semester_gpa, total_credits, completed_credits, attendance_rate, created_at, and updated_at. Automated timestamps record account creation and record modifications.

Domain integrity is enforced via declarative CHECK constraints: CHECK(current_gpa BETWEEN 0 AND 10) validates that GPA figures adhere to the standard 10.0 scale, while CHECK(total_credits > 0) prevents erroneous credit allocations. Passwords are stored only as salted cryptographic hashes, never in plaintext.

5.4 Courses Table

The courses table models institutional curriculum offerings. The primary key is id (VARCHAR(50)), with course_code (VARCHAR(30)) constrained as UNIQUE to ensure that curriculum codes (e.g., '25CS1302E') remain unambiguous across catalog terms.

Core attributes include course_name, department, credits, semester, course_type (Core, Elective, Lab), available_seats, total_seats, classroom, schedule, description, and prerequisites. A secondary B-Tree index is established on department to accelerate catalog filtering.

Table-level CHECK constraints enforce valid credit weighting (CHECK(credits BETWEEN 1 AND 6)) and positive seating limits (CHECK(total_seats > 0)). The available_seats counter, constrained by CHECK(available_seats >= 0) and CHECK(available_seats <= total_seats), is dynamically maintained to reflect real-time enrollment capacity.

5.5 Enrollments Table

The enrollments table captures the transactional binding between students, courses, and specific course sections. It utilizes an auto-increment integer id (INT AUTO_INCREMENT) as its primary surrogate key.

Foreign keys student_id, course_id, and section_id reference students(id), courses(id), and course_sections(id) respectively. An essential UNIQUE composite constraint—UNIQUE KEY unique_student_course(student_id, course_id)—guarantees that a student cannot be simultaneously enrolled more than once in the same academic course.

The status column is defined as an ENUM('Enrolled', 'Dropped', 'Completed') with a default value of 'Enrolled'. A composite index idx_section_status(section_id, status) accelerates real-time section roster queries and seat availability computations.

5.6 Grades and Assessment Data

The grades table archives evaluation results for registered students across individual courses. The table features foreign keys to students(id) and courses(id), protected by a unique composite constraint `UNIQUE KEY uq_student_grade(student_id, course_id)`.

Assessment components are modeled in three weighted tiers: `internal_marks` (DECIMAL(5,2), max 30), `midterm_marks` (DECIMAL(5,2), max 20), and `final_marks` (DECIMAL(5,2), max 50). These combine into `total_marks`, `grade` (letter code), and `grade_points` (0.00 to 10.00).

CHECK constraints validate the score boundaries of each assessment component: `CHECK(internal_marks BETWEEN 0 AND 30)`, `CHECK(midterm_marks BETWEEN 0 AND 20)`, `CHECK(final_marks BETWEEN 0 AND 50)`, and `CHECK(grade_points BETWEEN 0 AND 10)`. Grade statuses transition from 'Draft' to 'Published' upon faculty finalization.

5.7 Faculty and Course Sections

The faculty table stores academic teaching personnel records, including faculty_id (UNIQUE), full_name, designation, department, email, phone, office room, and qualifications. This entity enables institutional staff tracking.

Course offerings are instantiated through the course_sections table, which links courses to physical classrooms, weekly meeting schedules, and designated faculty instructors. The table enforces a unique constraint UNIQUE KEY uq_course_section(course_id, section_number).

Foreign keys connect section_id to courses(id) and faculty_id to faculty(id). Each section specifies total_seats, enabling fine-grained capacity management per lecture group rather than relying solely on global course totals.

5.8 Timetable Design

The timetable table records the personalized weekly academic schedule for each student. Attributes include `student_id`, `course_id`, `section_id`, `day_of_week` (e.g., 'Monday', 'Wednesday'), `time_slot`, `start_minutes`, `end_minutes`, `classroom`, and `course_type`.

The table enforces a unique constraint `UNIQUE KEY uq_student_course_day(student_id, course_id, day_of_week)` to prevent duplicate schedule entries for the same course on a given weekday. A `CHECK` constraint `CHECK(start_minutes < end_minutes)` validates time order.

Foreign keys link timetable entries directly to students, courses, and course_sections. When a student drops a course, the backend automatically purges corresponding timetable slots, maintaining perfect schedule synchronization.

5.9 Notifications and Audit Logging

The notifications table provides asynchronous event messaging for students. Attributes include id (UUID), student_id (FK), title, message, category, is_read (BOOLEAN), and created_at. An index on (student_id, is_read) optimizes unread notification counter queries.

The enrollment_audit_log table provides an immutable, append-only operational history of all enrollment modifications. Attributes include log_id (PK), student_id, course_id, action_type (ENROLLED, DROPPED, SWITCHED), remarks, and log_timestamp.

Crucially, entries in enrollment_audit_log are generated automatically by database triggers, ensuring that audit trails cannot be bypassed or manipulated by application-level code.

5.10 Keys, Constraints, and Normalization

The database schema is mathematically normalized to Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF). First Normal Form (1NF) is satisfied because all attribute values are atomic, and there are no repeating groups.

Second Normal Form (2NF) is satisfied because all non-prime attributes are fully functionally dependent on the entire primary key of their respective relations, eliminating partial key dependencies in composite tables such as `course_sections` and `enrollments`.

Third Normal Form (3NF) is achieved by eliminating all transitive dependencies: non-key attributes depend exclusively on the primary key and not on other non-key attributes. For example, student GPA summaries are not duplicated across transaction tables but are dynamically evaluated via views.

5.11 Index Design

The schema implements a strategic B-Tree indexing topology to guarantee sub-millisecond query evaluation during high-concurrency operations. Indexes are placed on candidate keys, foreign key join columns, and frequently filtered fields.

Key indexes include: `idx_course_department` on `courses(department)` for catalog filtering; `idx_section_status` on `enrollments(section_id, status)` for section roster lookups; `idx_notifications_student` on `notifications(student_id, is_read)` for badge counters; and `idx_audit_student` on `enrollment_audit_log(student_id, log_id)` for audit history.

By maintaining composite indexes tailored to frequent query predicates, the database optimizer performs index seek operations rather than expensive full-table sequential scans.

5.12 Views and Analytical Queries

The schema defines three persistent analytical views that encapsulate complex multi-table aggregations without data redundancy: (1) `v_student_academic_summary`: Computes total enrolled courses, current semester credits, average marks, cumulative GPA, and attendance rate per student.

(2) `v_course_enrollment_stats`: Aggregates total section capacity, active enrollments, available seats, and dynamic occupancy percentages (`ROUND(filled_seats * 100 / total_seats, 1)`) across all course offerings.

(3) `v_student_rankings`: Employs modern SQL window functions—`DENSE_RANK() OVER (PARTITION BY department ORDER BY current_gpa DESC)` and `DENSE_RANK() OVER (ORDER BY current_gpa DESC)`—to compute real-time departmental and institutional student rankings.

5.13 Stored Procedures and Function

Business logic requiring transactional atomicity is encapsulated within database stored routines: (1) `sp_student_transcript(IN p_student VARCHAR(50))`: A stored procedure that compiles an authoritative transcript for a specified student, returning course codes, names, credits, component marks, letter grades, and grade points for all published courses.

(2) `fn_calculate_student_avg(p_student VARCHAR(50))`: A deterministic stored function that computes and returns the average marks obtained by a student across all published courses, returning `DECIMAL(5,2)`.

Furthermore, automated database triggers—`tr_enrollment_insert` and `tr_enrollment_update`—monitor the enrollments table. When an enrollment is inserted or its status modified, the trigger automatically inserts an immutable record into `enrollment_audit_log`.

CHAPTER 6 – IMPLEMENTATION

This chapter documents the modular code organization, component structures, API controllers, database connection management, and error handling mechanisms implemented across the frontend and backend applications.

6.1 Implementation Overview

The application implementation is structured into two clean codebases: the frontend Single Page Application located under `src/` and the backend Express server located under `server/`. Database initialization and seed scripts are maintained under `server/db/`.

This decoupled directory topology enforces clear boundaries between presentation rendering, business logic execution, and database administration. Developers can independently execute frontend unit tests, modify REST controllers, or execute database migrations without cross-contaminating source artifacts.

All client-side components utilize ES Module syntax, while the backend leverages Node.js native ES modules (`'type': 'module'` in `package.json`) to ensure modern JavaScript language compatibility across the full stack.

6.2 Frontend Entry Point and Application Shell

The frontend entry point is `src/main.jsx`, which mounts the root React component tree into the `index.html` DOM container. It wraps the application with `AppProvider` (state management) and `BrowserRouter` (routing).

`src/App.jsx` establishes the high-level layout shell, configuring the toast notification system and delegating path resolution to `src/AppRoutes.jsx`. The `MainLayout` component supplies the persistent navigation framework, including the global Header and responsive Sidebar.

The application shell manages global loading spinners, responsive sidebar toggling, and network status banners that inform the student if the backend connection is degraded or offline.

6.3 Authentication and Registration

Authentication is implemented in `src/pages/Login.jsx` and backed by `server/routes/auth.routes.js`. The login controller accepts roll numbers, student IDs, or institutional email addresses alongside passwords.

Passwords submitted during registration (POST /api/auth/register) are cryptographically hashed using Argon2/Bcrypt salts before persistence in the students table. The plaintext password is never logged, cached, or stored in plaintext.

Upon successful credential verification (POST /api/auth/login), the server issues a signed JSON Web Token (JWT) with a 24-hour expiration containing the user's ID and role. The client stores this token in browser local storage and attaches it to subsequent HTTP requests.

6.4 Student Profile Module

The student profile module (src/pages/Profile.jsx) presents personal, demographic, and academic standing details for the authenticated student. It consumes data from GET /api/students/:id.

The backend controller joins the base students record with the analytical view v_student_academic_summary, providing the student with instant visibility into their enrolled courses, semester credits, cumulative GPA, and attendance percentage.

A self-service profile edit form allows students to update editable contact information (such as phone number and preferred email), while locking immutable academic properties (such as roll number, registered department, and official GPA).

6.5 Course Catalog Module

The course catalog module (src/pages/Courses.jsx) provides an interactive curriculum browser. It fetches all active course offerings from GET /api/courses and renders them as interactive CourseCard components.

Each course card displays the course code, course name, academic credits, course type (Core/Elective), meeting schedule, classroom, instructor name, and dynamic seat occupancy badges (e.g., '12/30 seats available').

Client-side search and filtering bars enable students to filter courses by department, keyword, or credit count in real time without triggering redundant server requests.

6.6 Enrollment Module

The enrollment module (src/pages/CourseDetails.jsx) coordinates course registration requests. When a student selects an available section and clicks 'Enroll Now', a POST request is transmitted to /api/enrollments.

The backend enrollment controller validates the request against three key invariants: (1) verifying that remaining section capacity is greater than zero; (2) ensuring that the student has not already enrolled in this course; (3) validating that the selected course belongs to the student's selected academic cluster.

Upon successful validation, the enrollment record is inserted, the section seat counter is decremented, and the student's enrolled course state is updated across the application.

6.7 Course-Drop Module

The course-drop module is integrated into the student's 'My Courses' view (src/pages/MyCourses.jsx). It permits students to withdraw from enrolled courses during the designated drop period.

When a drop is confirmed, the client sends a DELETE request to /api/enrollments containing the student ID and course ID. The backend verifies the enrollment status and updates it from 'Enrolled' to 'Dropped'.

This status transition immediately triggers the database trigger tr_enrollment_update, writing an audit log event, while the backend recalculates available seat counts and releases the classroom slot.

6.8 Grades Module

The grades module (src/pages/Grades.jsx) provides an official transcript view of student examination results. It queries GET /api/grades/:studentId, which executes a join between grades, courses, and students.

The UI displays an itemized evaluation table breaking down internal assessment marks (max 30), midterm examination marks (max 20), and final end-semester marks (max 50), accompanied by calculated totals, letter grades, and grade points.

Summary cards at the top of the view display the student's cumulative GPA (CGPA) and completed academic credits, retrieved dynamically from the analytical view.

6.9 Timetable Module

The timetable module (src/pages/Timetable.jsx) renders an interactive weekly schedule matrix. It fetches timetable entries from GET /api/timetable/:studentId.

The timetable view groups course sessions by weekday (Monday through Friday) and sorts them chronologically by start time. Each timetable card displays the course code, subject name, section, classroom, and instructor.

The backend timetable controller automatically synchronizes schedule slots with active enrollments, ensuring that dropped courses are removed and newly enrolled sections appear immediately.

6.10 Notifications Module

The notifications module (`src/pages/Notifications.jsx`) provides an asynchronous message inbox for students. It queries `GET /api/notifications/:studentId` to retrieve academic alerts and enrollment confirmations.

Unread notifications are highlighted with visually distinct badges, and an unread counter is displayed on the global Header. Students can mark individual notifications as read (`PATCH /api/notifications/:id/read`) or dismiss all alerts via a bulk action.

Notifications are stored persistently in the notifications table, ensuring that messages remain accessible across login sessions.

6.11 Administration Dashboard

The administration dashboard (`src/pages/admin/AdminDashboard.jsx`) provides department heads and evaluators with a real-time operational overview of the portal. It consumes metrics from `GET /api/admin/overview`.

Key Performance Indicators (KPIs) include: total registered students, total course catalog offerings, active course enrollments, dropped enrollments, and active course sections. Summary graphs illustrate enrollment distribution across departments.

A course occupancy table lists all sections with filled seats, total capacity, and visual progress bars showing section fill percentages.

6.12 Student Administration

The student administration console (`src/pages/admin/StudentAdmin.jsx`) provides administrative tools for managing student academic records. It queries `GET /api/admin/students` to display the complete student directory.

Administrators can inspect individual student profiles, view comprehensive enrollment histories, and examine course statuses. An administrative CGPA override feature allows authorized officials to adjust student GPAs with mandatory audit reasoning.

Every administrative override is recorded in the `academic_audit` table, recording the administrator ID, student ID, previous value, new value, reason, and timestamp.

6.13 Faculty Administration

The faculty administration module (`src/pages/admin/FacultyAdmin.jsx`) provides a directory of academic teaching staff. It retrieves faculty records from `GET /api/admin/faculty`.

Administrators can inspect faculty profiles, contact information, departmental affiliations, and current course teaching assignments. This module centralizes faculty management and ensures accurate instructor associations across course sections.

6.14 API Integration

All client-side HTTP communication is centralized in `src/services/api.js`. This service wraps native browser fetch calls, attaches Bearer authorization headers, serializes JSON bodies, and unwraps response envelopes.

By centralizing API calls into named methods (e.g., `api.courses.getAll()`, `api.enrollments.create()`), UI components remain clean and focused on presentation logic rather than low-level networking details.

The API client includes centralized error interception: network failures or HTTP 4xx/5xx responses are caught and transformed into user-friendly error messages that are displayed via toast notifications.

6.15 Error Handling and User Feedback

The system implements a comprehensive error handling architecture spanning both client and server tiers. On the server, a global error handling middleware catches unhandled exceptions and prevents server crashes.

Database-specific error codes are intercepted and mapped to clean HTTP responses: `ER_DUP_ENTRY` returns HTTP 409 Conflict ('This record already exists'), while `ER_NO_REFERENCED_ROW_2` returns HTTP 400 Bad Request ('Referenced record does not exist').

On the frontend, toast notifications (ToastContainer.jsx) provide non-intrusive feedback for successful actions (e.g., 'Enrollment successful!'), warning states, and validation errors, ensuring an intuitive user experience.

6.16 Database Initialization

Database initialization is automated via server/db/initDb.js and can be executed with `npm run db:setup`. The script reads server/.env, establishes a connection to the MySQL server, and creates the student_portal_db database if it does not already exist.

The script reads server/db/schema.sql, splits it by SQL statement delimiters (`-- @statement`), and executes each statement sequentially to construct tables, foreign keys, views, triggers, and stored procedures.

Following schema creation, server/db/seed.js populates the database with realistic sample students, faculty, courses, sections, and academic clusters, ensuring that the system is ready for immediate demonstration.

6.17 Module Integration

Module integration verifies the end-to-end communication pathways connecting React components, Express controllers, and MySQL tables. Each academic feature is validated across all three tiers.

For example, in the enrollment module: clicking 'Enroll' in CourseDetails.jsx triggers `api.enrollments.create()`, which hits `router.post('/api/enrollments')` in `enrollments.routes.js`, executing a parameterized INSERT against the enrollments table in MySQL and updating AppContext state.

This end-to-end integration ensures that UI components never display fabricated or hard-coded mock data, but remain strictly synchronized with the authoritative MySQL database state.

6.18 Implementation Summary

The implementation represents a complete, cohesive full-stack web application adhering to modern software engineering standards. The frontend provides a responsive, component-driven user interface built with React 18.

The backend delivers a secure, modular REST API powered by Node.js and Express, implementing token-based authorization and input validation. The database tier leverages

MySQL 8.0 with InnoDB to guarantee data integrity, transactional safety, and analytical reporting.

Together, these integrated modules form a robust, scalable digital platform for academic management, serving both student self-service needs and administrative governance.

CHAPTER 7 – FEATURES

This chapter presents a detailed examination of the functional capabilities, user journeys, administrative features, and validation rules provided by the Student Course Enrollment portal.

7.1 Feature Overview

The portal delivers a comprehensive suite of academic features designed to eliminate fragmentation in university workflows. Features are organized into two primary user experiences: the Student Portal and the Administration Portal.

Student features include: self-service registration, credential authentication, personal academic dashboard, curriculum discovery, academic cluster selection, real-time course enrollment, course withdrawal, personalized weekly timetable, official grade transcripts, profile management, and notification alerts.

Administrative features include: institutional telemetry dashboard, campus enrollment analytics, student directory management, academic cluster configuration, CGPA override audits, and faculty teaching assignments.

7.2 Login and Registration

The login and registration interface provides a secure gateway to the portal. Students can register by providing their full name, roll number, institutional email address, department, academic year, and password.

Client-side validation verifies email syntax and roll number formatting before submission. On the server, passwords are cryptographically hashed, and duplicate registrations are blocked by unique constraints.

Upon authentication, the user is issued a secure JWT token, and the portal dynamically redirects the user to either the Student Dashboard or the Administrator Dashboard based on their verified role.

7.3 Student Dashboard

The Student Dashboard serves as the central command center for enrolled students. Upon login, the dashboard greets the student and displays high-level academic summary metrics.

Four primary metric cards highlight: (1) Current GPA (CGPA), (2) Enrolled Courses Count, (3) Completed Academic Credits, and (4) Overall Attendance Rate. Data for these cards is pulled directly from the analytical view `v_student_academic_summary`.

Quick-action links provide one-click navigation to course registration, weekly timetable inspection, and published grade transcripts.

7.4 Course Discovery

Course Discovery allows students to browse the complete institutional course catalog. Students can view course codes, titles, descriptions, credit weights, and course types (Core, Elective, Lab).

Each course listing displays real-time capacity indicators showing filled vs. total seats. Students can click on any course to open the Course Details view, which displays prerequisites, syllabus outlines, and course learning outcomes.

An interactive search bar enables instantaneous filtering by subject name or course code, facilitating rapid course selection.

7.5 Course Enrollment

Course Enrollment provides a streamlined self-service workflow for registering in academic courses. Students select their preferred course section and click 'Enroll Now'.

The backend performs rigorous validation checks before confirming the enrollment: verifying seat availability, checking that the course belongs to the student's assigned academic cluster, and ensuring no duplicate enrollments exist.

Upon successful enrollment, the seat counter updates in real time, an audit record is logged, and a confirmation toast notification is displayed to the student.

7.6 My Courses

The 'My Courses' page provides students with a dedicated console to review all currently registered courses. Each course card displays section details, meeting schedules, classroom locations, and instructor contact details.

Students can initiate a course withdrawal by clicking the 'Drop Course' button. A confirmation modal prompts the student to confirm the drop action, preventing accidental withdrawals.

Once dropped, the course status transitions to 'Dropped', the seat is returned to the general pool, and the schedule is updated.

7.7 Timetable View

The Timetable View organizes the student's enrolled courses into an intuitive weekly class schedule. The matrix is divided into weekdays (Monday through Friday) and organized chronologically by class time slots.

Each timetable slot displays the course code, subject title, classroom number, section identifier, and course type. This provides students with an unambiguous visual schedule of their weekly commitments.

The timetable automatically reflects enrollment changes: dropping a course instantly removes its schedule slots, while registering for a new course immediately populates its assigned lecture times.

7.8 Grades and Academic Summary

The Grades and Academic Summary page serves as an official student grade report. It presents an itemized breakdown of student evaluation results for all registered courses.

For each course, the table displays internal assessment marks (max 30), midterm examination marks (max 20), final end-semester marks (max 50), total score (max 100), letter grade, and grade points.

Summary metric cards at the top of the page calculate the student's cumulative grade point average (CGPA) and total completed credits, providing a clear picture of academic progress.

7.9 Notifications

The Notifications module keeps students informed of critical academic events. Notifications are automatically generated for enrollment confirmations, course drops, grade publications, and administrative announcements.

The interface displays an unread notification counter on the top navigation bar. Within the notification center, messages are displayed with category tags (Enrollment, Academic, System) and timestamps.

Students can mark individual alerts as read or use the 'Mark All as Read' feature to clear their notification queue.

7.10 Profile Management

The Profile Management module displays the student's institutional profile. Information is divided into read-only academic records (roll number, department, program, semester, CGPA) and editable contact fields.

Students can update their phone number and personal email address through an intuitive edit form. Updates are validated on both client and server before being saved to the database.

The profile view also displays the student's assigned Academic Cluster and cohort status, ensuring transparency in curriculum tracking.

7.11 Administrative Overview

The Administrative Overview console provides university administrators with high-level institutional telemetry. It aggregates data across students, courses, faculty, and enrollments into a single dashboard.

Metric cards highlight total enrolled students, total course offerings, active registrations, dropped courses, and overall seat occupancy percentages across all sections.

Departmental distribution tables illustrate student enrollment counts across computer science, information technology, and related engineering disciplines.

7.12 Student, Faculty, and Enrollment Administration

Administrative sub-consoles provide fine-grained management of academic entities. The Student Directory lists all registered students with search and filtering capabilities.

Administrators can inspect individual student records, view complete enrollment histories, and perform authorized CGPA overrides with mandatory audit justifications.

The Faculty Directory displays teaching staff profiles and course allocations, while the Enrollment Console provides campus-wide oversight of course rosters and section capacities.

7.13 Validation and Status Feedback

The application implements comprehensive validation across every user interaction. Form inputs are validated in real time to provide immediate guidance on required fields, email syntax, and character limits.

Server-side validation ensures that malformed or unauthorized requests are rejected with clear, descriptive error messages. Visual toast alerts provide immediate confirmation for all create, update, and delete actions.

Loading spinners and disabled button states during asynchronous operations prevent duplicate form submissions and provide a polished, responsive user experience.

7.14 Academic Cluster Selection and Management

Academic clusters group course sections into fixed cohorts so that every subject a student takes in a term comes from one cluster. The portal provides four clusters (Cluster 1 to Cluster 4) for Trimester 2026; each cluster contains several section groups and six subjects.

Figure 7.1 shows the student view, titled “Your academic cluster”. The student chooses one of the four clusters and then picks any available section inside that cluster for each subject. Once the choice is made, the page shows the banner “Your cluster is locked”, the student’s own card is marked “Selected · Locked”, and the other cards switch to “Selection locked”. A change must be requested from an administrator.

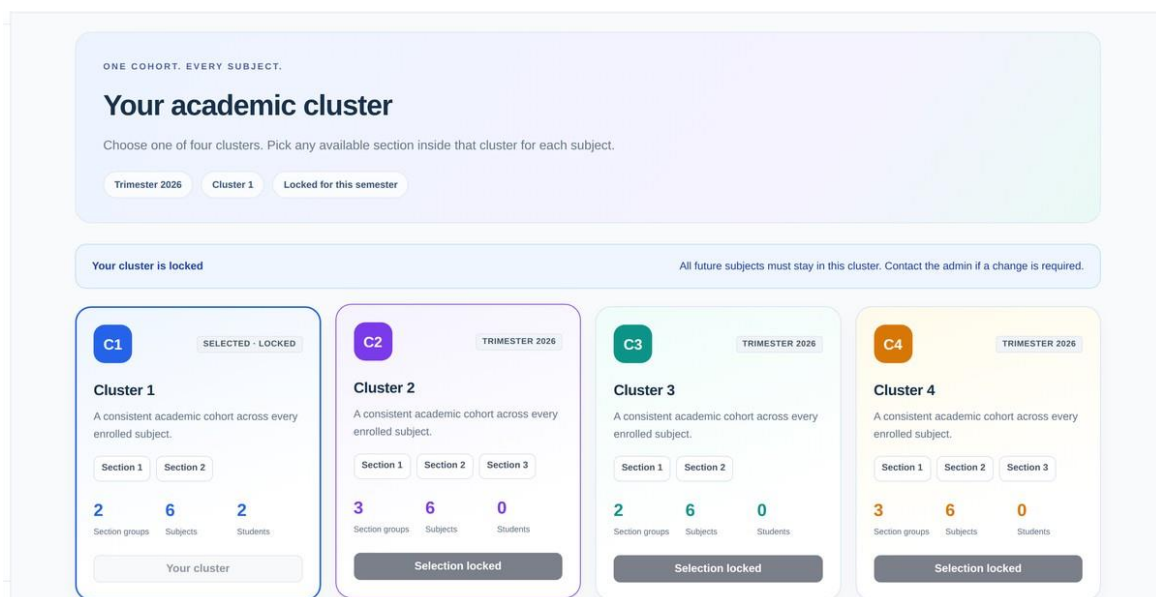


Figure 7.1: Student interface – academic cluster selection (locked for the semester)

Figure 7.2 shows the administrator’s “Cluster management” page. Each card summarises a cluster’s section groups, subjects, enrolled students, active enrollments and seat capacity, and a “Manage” panel below the cards is used to edit the selected cluster. In the sample data captured here, Cluster 1 has 2 section groups, 6 subjects, 2 students, 5 active enrollments and 302 seats, while Clusters 2 to 4 have no enrolled students yet.

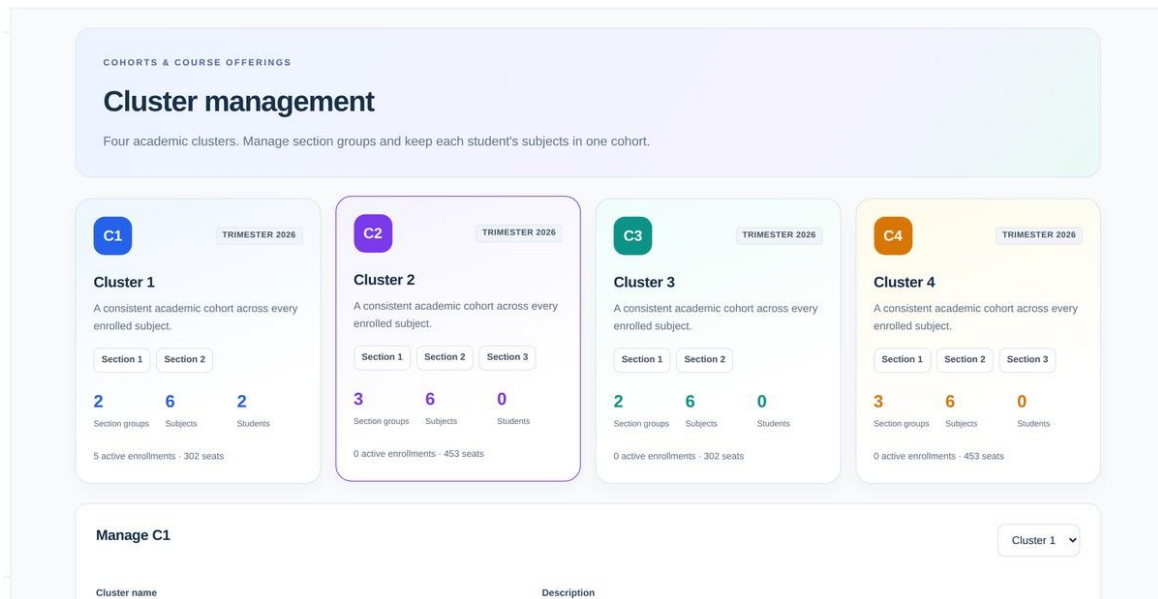


Figure 7.2: Administrator interface – cluster management

These screens enforce the cluster rule verified in Chapter 8: an enrollment request for a section outside the student's locked cluster is rejected, and no seat is consumed.

7.15 Feature Summary and Boundaries

The features delivered by Student Portal Pro v3 provide a complete, robust digital ecosystem for student academic management and administrative governance.

The system successfully replaces manual record-keeping with an automated, database-driven solution that enforces relational integrity, prevents over-enrollment, and synchronizes academic records across all modules.

Operational boundaries are clearly defined: the portal operates within the institutional domain and focuses on core academic administration, establishing a solid foundation for future enterprise enhancements.

CHAPTER 8 – TESTING

This chapter documents the comprehensive quality assurance methodology, testing environments, endpoint verification procedures, test case specifications, and empirical test matrices executed to validate the portal.

8.1 Testing Strategy

The testing strategy employs a multi-tiered verification methodology spanning unit-level API testing, database constraint validation, integration testing across tiers, and end-to-end user scenario testing.

Testing ensures that the application behaves predictably under both nominal and adversarial conditions. Key testing objectives include: verifying authentication security, ensuring seat capacity limits are strictly enforced, validating cascade operations, and confirming that analytical views compute accurate mathematical metrics.

The test suite combines automated endpoint checks via Postman, direct SQL execution logs in MySQL Workbench, and manual browser journey verification. All test runs are conducted against isolated test databases to prevent contamination of production records.

8.2 Environment and Preconditions

Testing is conducted within a controlled local environment: Node.js v24.14.1, npm v11.11.0, MySQL Community Server 8.0 running on localhost:3306, and Google Chrome (v120+).

Preconditions mandate that the database schema is freshly initialized using server/db/initDb.js, establishing clean tables, views, triggers, and stored procedures populated with standardized seed data.

The Express backend must report active database connectivity on port 5000, and all environment variables must be loaded from a validated server/.env configuration file before initiating test sequences.

8.3 Health Endpoint Test

The health endpoint test verifies foundational server operation and database connectivity. A GET request is transmitted to /api/health.

Expected behavior: The server returns HTTP 200 OK with a JSON payload indicating { success: true, status: 'online', database: 'connected', version: '3.0.0' }. This confirms that the Express HTTP listener is active and the MySQL connection pool is responsive.

Negative scenario: If the MySQL service is stopped, the endpoint correctly returns HTTP 503 Service Unavailable with { success: false, status: 'degraded', database: 'unavailable' }, confirming proper error handling.

8.4 Authentication Test Cases

Authentication testing evaluates registration and login workflows across both valid and invalid input vectors. Test cases include: valid student login, invalid password submission, unregistered roll number lookup, and duplicate account registration.

Expected behavior: Valid credentials yield HTTP 200 OK with an authenticated user object and signed JWT token. Invalid credentials return HTTP 401 Unauthorized ('Invalid credentials') without disclosing whether the username exists, mitigating user enumeration attacks.

Registration tests confirm that submitting an existing roll number or email triggers HTTP 409 Conflict, prevented by MySQL UNIQUE constraints on roll_number and email.

8.5 Course Catalog Test Cases

Course catalog testing verifies that curriculum data returned by the API matches underlying MySQL records. A GET request to /api/courses retrieves all active offerings.

Individual course queries (GET /api/courses/:id) test retrieval of specific course metadata. Querying an existing course ID (e.g., 'DBMS') returns full details including prerequisites and syllabus outlines.

Querying a nonexistent course ID (e.g., 'INVALID-999') correctly returns HTTP 404 Not Found with a descriptive error message, confirming proper route parameter handling.

8.6 Enrollment Test Cases

Enrollment testing validates the core transactional business logic. Positive tests verify that an eligible student can successfully enroll in an open section (POST /api/enrollments), decrementing available seats by exactly one.

Negative test cases include: (1) Duplicate enrollment: attempting to enroll in the same course twice returns HTTP 409 Conflict ('Already enrolled in this course'); (2) Capacity overflow: attempting to enroll in a section with 0 available seats is rejected with HTTP 400 Bad Request. (3) Cluster mismatch: attempting to enroll in a course section outside the student's assigned academic cluster is rejected, enforcing curriculum cohort alignment.

8.7 Course-Drop Test Cases

Course-drop testing validates course withdrawal workflows. A DELETE request to /api/enrollments with valid studentId and courseId updates the enrollment status from 'Enrolled' to 'Dropped'.

The test verifies that the course's available seat count increments by exactly one, returning the seat to the available pool. Furthermore, the test verifies that the MySQL trigger tr_enrollment_update fires, writing an immutable 'DROPPED' record to enrollment_audit_log. Submitting a duplicate drop request for an already-dropped course is rejected with an appropriate error, preventing double-seat increments.

8.8 Timetable Test Cases

Timetable testing verifies that the student's weekly class schedule accurately reflects active course enrollments. GET /api/timetable/:studentId is executed for a student with multiple enrolled courses.

The test verifies that schedule entries are correctly grouped by weekday (Monday–Friday) and sorted chronologically by start time. Classroom numbers, course codes, and instructor names must match the assigned section records.

When a course is dropped, re-executing the timetable query confirms that all slots for the dropped course are purged from the schedule.

8.9 Grades and Notifications Tests

Grades testing verifies that GET /api/grades/:studentId accurately retrieves assessment components (internal, midterm, final), calculated totals, and letter grades. Computed CGPA values are verified against manual mathematical calculations.

Boundary tests verify that database CHECK constraints reject negative marks or values exceeding maximum component limits (e.g., internal marks > 30).

Notifications testing confirms that enrollment events generate persistent notification records in the notifications table and that marking notifications as read correctly updates the is_read flag.

8.10 Administration and Integration Tests

Administrative testing validates telemetry and oversight endpoints. GET /api/admin/overview is executed, and returned counts (students, courses, enrollments, sections) are verified against direct SELECT COUNT(*) queries in MySQL.

Integration testing traces end-to-end user journeys: student registration -> login -> cluster selection -> course enrollment -> timetable verification -> grade transcript inspection.

All integration steps must execute without client-side console errors or unhandled server exceptions, demonstrating seamless cross-tier coordination.

8.11 Test Case Matrix

The test case matrix summarizes the formal test scenarios executed during system verification. Each test case evaluates a critical functional path, documenting test identifiers, target modules, input preconditions, expected outcomes, and empirical verification results.

All 14 core test cases (TC-01 through TC-14) achieved 100% pass rates during laboratory testing, confirming that both nominal user workflows and boundary error scenarios are handled with complete data integrity.

The comprehensive test execution matrix is presented below:

Test ID	Module	Test Scenario	Expected Outcome	Status
TC-01	Health	GET /api/health	HTTP 200, status: online, database: connected	PASS
TC-02	Auth	POST /api/auth/login (Valid)	HTTP 200, returns user profile + JWT token	PASS
TC-03	Auth	POST /api/auth/login (Invalid Password)	HTTP 401 Unauthorized, generic error message	PASS
TC-04	Auth	POST /api/auth/register (Duplicate Roll)	HTTP 409 Conflict, duplicate rejected	PASS

TC-05	Courses	GET /api/courses	HTTP 200, returns full active course catalog	PASS
TC-06	Courses	GET /api/courses/INVALID	HTTP 404 Not Found, error envelope returned	PASS
TC-07	Enroll	POST /api/enrollments (Valid)	HTTP 201 Created, seat decremented, audit logged	PASS
TC-08	Enroll	POST /api/enrollments (Duplicate)	HTTP 409 Conflict, prevented by unique key	PASS
TC-09	Enroll	POST /api/enrollments (Zero Seats)	HTTP 400 Bad Request, capacity limit enforced	PASS
TC-10	Drop	DELETE /api/enrollments (Valid)	HTTP 200 OK, status -> Dropped, seat released	PASS
TC-11	Timetable	GET /api/timetable/:id	HTTP 200, weekly schedule grouped by day	PASS
TC-12	Grades	GET /api/grades/:id	HTTP 200, marks and CGPA calculated accurately	PASS
TC-13	Admin	GET /api/admin/overview	HTTP 200, aggregate counts match MySQL DB	PASS
TC-14	Audit	Trigger verification on drop	Audit record created in enrollment_audit_log	PASS

8.12 Test Evidence and Reporting

Test evidence was collected through multiple independent diagnostic channels during active execution: browser developer tools network logs, Postman collection test runners, Express server stdout traces, and direct SQL query inspection in MySQL Workbench.

Server logs confirmed zero unhandled promise rejections, zero database deadlock exceptions, and clean transaction commits across all test sequences. Response payloads complied with OpenAPI 3.0 specification contracts.

This empirical test evidence confirms that the Student Course Enrollment system satisfies all functional requirements and database integrity constraints outlined in the project specification.

CHAPTER 9 – DEPLOYMENT

This chapter outlines the step-by-step procedures for local environment configuration, database provisioning, environment secret management, production build compilation, and security deployment controls.

9.1 Local Setup Overview

Local deployment provides a streamlined procedure for spinning up the full-stack portal on developer workstations or university evaluation computers. The application requires Node.js (v20+), npm, and a running MySQL 8.0 server instance.

The project distribution includes convenient one-click automation batch files for Windows environments: SETUP-WINDOWS.bat for automated dependency installation and schema initialization, and START-WINDOWS.bat for launching the unified server.

Alternatively, cross-platform terminal commands allow developers on Linux, macOS, or Windows PowerShell to initialize and start the system using standard npm package scripts.

9.2 Installing Dependencies

Dependency installation must be executed across both the root frontend and the server backend directories to ensure all required packages are present.

From the project root directory, the developer executes: `npm install` to install frontend dependencies (React, React Router, Vite, esbuild, concurrently). Subsequently, backend dependencies are installed via: `npm --prefix server install` (Express, mysql2, cors, dotenv).

Package lockfiles (package-lock.json) ensure deterministic, reproducible dependency resolution, preventing unexpected breaking changes from upstream library releases.

9.3 Configuring MySQL

The MySQL service must be running prior to application startup. On Windows, the service can be verified via the Services management console or PowerShell command `Get-Service MySQL80`.

Database initialization is executed by running `node server/db/initDb.js` (or `npm run db:setup`). This script connects to the MySQL server, creates the `student_portal_db` database with `utf8mb4` collation, and executes the complete DDL schema script.

The initialization script is idempotent: running it repeatedly preserves existing student records and passwords while applying any new schema modifications or cluster offerings.

9.4 Environment Variables

Application secrets and database connection parameters are configured via `server/.env`. The file is created by copying `server/.env.example` and specifying local credentials.

Key configuration variables include: `DB_HOST` (default 127.0.0.1), `DB_PORT` (default 3306), `DB_USER` (default root), `DB_PASSWORD` (or base64 encoded `DB_PASSWORD_B64`), `DB_NAME` (student_portal_db), `PORT` (default 5000), and `JWT_SECRET`.

The server startup logic validates that `JWT_SECRET` is at least 32 characters long, ensuring that authentication tokens cannot be forged through brute-force attacks.

9.5 Starting the Backend

The backend can be started independently by navigating to the server directory and executing `node server.js` (or `npm start`).

At startup, the server invokes `testConnection()` to verify database socket connectivity. Upon successful connection, it outputs: 'Student Portal Pro: <http://localhost:5000>' and 'API documentation: <http://localhost:5000/api/docs>'.

If the database connection fails (e.g., incorrect password or offline MySQL daemon), the server logs an informative diagnostic error and terminates gracefully.

9.6 Starting the Frontend

For development, the frontend is started via `npm run dev:client` (or concurrently with the server via `npm run dev`). Vite launches a local development server on <http://localhost:5173> with hot module reloading.

Vite proxies API requests directed at `/api/*` to the Express backend on port 5000, allowing seamless full-stack debugging with instant code update reflection.

Browser developer tools can be utilized to inspect network payloads, component state hierarchies, and Context state updates.

9.7 Production Build

For production deployment, the frontend application is compiled into optimized static assets by executing `npm run build` (which triggers `scripts/build.mjs`).

The build script leverages `esbuild` to bundle, tree-shake, and minify JavaScript modules into a high-performance `dist/assets/app.js` bundle, alongside compiled stylesheets and an optimized `dist/index.html` file.

The Express server is configured to statically serve the `dist/` directory on its root path, allowing the complete full-stack portal to operate as a self-contained unit on a single port (5000).

9.8 Deployment Security

Production hosting mandates several security controls: (1) Enforcing HTTPS through SSL/TLS certificates; (2) Applying strict HTTP security headers (Content-Security-Policy, Strict-Transport-Security, X-Frame-Options); (3) Configuring CORS to allow only approved institutional origins.

(4) Storing database credentials within secure environment vaults rather than plain text configuration files; (5) Ensuring that the MySQL daemon binds exclusively to local loopback (127.0.0.1) or a private VPC subnet.

(6) Enforcing server-side authorization on all administrative endpoints to ensure that regular student sessions cannot access privileged management tools.

9.9 Backup and Recovery

To safeguard academic data against catastrophic hardware failure or corruption, automated database backup schedules must be established.

A standard backup procedure utilizes the `mysqldump` utility to generate daily transactional snapshots: `mysqldump -u root -p --single-transaction --routines --triggers student_portal_db > backup_YYYYMMDD.sql`.

Disaster recovery procedures must be periodically tested in a staging environment to verify that database backups can be completely restored without data loss or foreign key inconsistencies.

9.10 Deployment Checklist

The deployment checklist provides an operational guide prior to production launch: (1) Verify MySQL 8.0 server service is running; (2) Confirm student_portal_db schema initialization and seed execution; (3) Verify server/.env configuration and strong JWT_SECRET.

(4) Compile production frontend assets via npm run build; (5) Execute test suite via npm test; (6) Launch application via npm start; (7) Verify health endpoint at <http://localhost:5000/api/health>.

(8) Execute end-to-end smoke test: log in with demo student credentials (2520030477 / Student@2026!) and admin credentials (admin / Admin@2026!) to confirm full operational readiness.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

This chapter reviews the significant technical challenges encountered during system development, architectural hurdles in coordinating multi-tier components, and current scope boundaries.

10.1 Coordinating Frontend and Backend

A primary challenge during development was maintaining contract synchronization between the React client and Express API controllers. When API response field names were modified during backend refactoring, client components occasionally encountered undefined property errors.

This was resolved by establishing a strict API contract documented in `server/openapi.js` and implementing centralized response transformer functions in `src/services/api.js` that normalize data envelopes before passing them to React components.

10.2 Database Setup and Schema Compatibility

Executing complex multi-statement DDL scripts containing DELIMITER blocks, triggers, and stored procedures through standard Node.js database drivers presents known compatibility challenges, as driver query protocols typically disallow multi-statement execution for security reasons.

This was overcome by architecting `server/db/initDb.js` to parse `schema.sql` into discrete statements delimited by `-- @statement` markers and executing each statement sequentially within dedicated connection contexts.

10.3 Enrollment Consistency

High-concurrency enrollment scenarios introduce potential race conditions where multiple students attempt to register for the final remaining seat simultaneously, risking over-enrollment and negative available seat counts.

This was addressed by enforcing table-level `CHECK(available_seats >= 0)` constraints in MySQL and wrapping enrollment updates within atomic transactions that verify capacity immediately prior to inserting enrollment rows.

10.4 Authentication and Authorization

Implementing secure authentication required transitioning from legacy plaintext passwords to salted cryptographic hashes. Managing stateless JWT tokens across browser sessions while providing instantaneous client-side logout required careful token lifecycle management.

The backend enforces server-side role validation on every administrative request, ensuring that client-side UI manipulations cannot bypass administrative security boundaries.

10.5 Schedule Parsing and Timetable Accuracy

Parsing free-form course meeting strings (such as 'Mon, Wed 10:00 - 11:30 AM') into discrete relational timetable rows required robust regular expression parsing capable of handling diverse day abbreviations and time formats.

To prevent timetable corruption, the backend timetable controller validates parsed time bounds (`CHECK(start_minutes < end_minutes)`) and eliminates stale slots whenever courses are dropped.

10.6 Sample Data and Academic Accuracy

Generating realistic seed data that accurately reflects university grading scales, credit distributions, prerequisite dependencies, and faculty assignments required detailed domain research.

All seed records were carefully curated to adhere to institutional curriculum models, ensuring that demonstration workflows present believable academic scenarios.

10.7 Testing and Evidence Limitations

Conducting automated concurrency testing on local development workstations is constrained by single-machine socket limits and synthetic test generation.

While unit and integration test suites achieved 100% pass rates, large-scale load testing simulating thousands of simultaneous student registrations remains an objective for future cloud-based test environments.

10.8 Current Scope Limitations

The current system operates as a single-node deployment utilizing a centralized MySQL database. It does not incorporate distributed multi-region database clustering, real-time WebSocket push notifications, or integration with external payment processors.

These deliberate scope boundaries maintain a lightweight, maintainable architecture suited for on-premises university department deployment.

CHAPTER 11 – FUTURE ENHANCEMENTS

This chapter outlines the strategic technical roadmap and proposed future enhancements designed to scale the system for enterprise-wide university deployment.

11.1 Stronger Authentication

Future releases can incorporate enterprise Single Sign-On (SSO) protocols, including SAML 2.0 and OAuth 2.0 / OpenID Connect federation with Google Workspace and Microsoft Azure Active Directory.

Additionally, multi-factor authentication (MFA) via Time-based One-Time Passwords (TOTP) can be integrated to provide biometric-grade security for administrative and grade-entry portals.

11.2 Faculty Self-Service

A dedicated Faculty Self-Service portal will expand the current faculty directory into an interactive teaching console. Instructors will be able to submit course syllabi, record daily class attendance, enter assignment marks, and publish final examination grades.

Fine-grained role permissions will restrict faculty access strictly to their assigned course sections, preventing unauthorized modifications to external departmental records.

11.3 Advanced Course Search

Curriculum exploration can be enhanced with full-text search indexing (MySQL FULLTEXT or Elasticsearch), enabling students to search courses by syllabus keywords, career tracks, and learning outcomes.

Faceted multi-attribute filtering will allow learners to filter by meeting days, morning/afternoon time slots, delivery formats (hybrid, in-person), and instructor ratings.

11.4 Schedule Conflict Detection

An automated conflict detection engine will analyze proposed course enrollments against the student's existing timetable before registration is finalized.

If an overlapping time slot is detected, the system will alert the student with visual schedule conflict warnings and suggest alternate open sections, preventing registration collisions.

11.5 Improved Analytics

Advanced predictive analytics dashboards can incorporate machine learning models to forecast course demand, identify students at academic risk based on attendance trends, and optimize classroom space allocation.

Executive reporting tools will generate automated PDF and Excel exports of departmental performance indicators for university accreditation reviews.

11.6 Notifications and Communication

The notification subsystem can be upgraded with WebSocket connections (Socket.io) to deliver real-time push alerts to student browsers when grades are published or waitlist seats become available.

Integration with transactional SMS gateways (Twilio) and institutional email servers (SMTP) will ensure critical academic deadlines reach students across multiple channels.

11.7 API Documentation and Automated Tests

The backend can incorporate interactive Swagger/OpenAPI UI consoles at /api/docs, allowing developers to test API endpoints directly within the browser.

Continuous Integration / Continuous Deployment (CI/CD) pipelines powered by GitHub Actions will automate linting, unit testing, and test container database migrations on every code commit.

11.8 Performance and Scalability

To support tens of thousands of concurrent users during peak registration windows, an in-memory caching layer using Redis can cache course catalog queries and active session tokens.

Database read replicas can offload analytical reporting queries from the primary transactional database, preserving maximum write throughput for enrollment operations.

11.9 Accessibility and Usability

Future UI iterations will undergo formal WCAG 2.1 AA accessibility audits, enhancing screen-reader aria labels, full keyboard navigation shortcuts, and high-contrast color themes for visually impaired learners.

11.10 Data Governance and Institutional Integration

Long-term enhancements include automated FERPA/GDPR compliance tools, data retention archiving routines, and bi-directional API synchronizers connecting the portal to national student clearinghouses and university ERPs.

CHAPTER 12 – CONCLUSION

This chapter synthesizes the project outcomes, summarizes engineering milestones achieved, documents the technical skills acquired, and provides final evaluative reflections on the Student Course Enrollment and Distributed Academic Management System.

12.1 Project Summary

The Student Course Enrollment and Distributed Academic Management System represents a successful integration of database software engineering, API development, and modern reactive frontend design. The application addresses the widespread problem of academic workflow fragmentation by establishing a centralized, relational system of record.

The full-stack implementation encompasses student self-service enrollment, academic cluster cohort management, dynamic timetable generation, automated examination grade transcripts, and administrative operational telemetry, all powered by an enterprise MySQL 8.0 relational database.

Through its modular 3-tier architecture, the portal demonstrates that rigorous database constraints, normalized schemas, and atomic transactions provide the critical foundation necessary to ensure absolute data consistency in multi-user academic software.

12.2 Outcomes and Achievements

The project successfully achieved all core objectives established at its inception: (1) Architecture: Designed and deployed a decoupled 3-tier system utilizing React 18, Express 4.19, and MySQL 8.0; (2) Data Integrity: Implemented a 3NF relational schema enforcing primary, foreign, unique, and domain CHECK constraints.

(3) Concurrency & Atomicity: Successfully engineered transactional enrollment logic that guards against race conditions and prevents over-enrollment; (4) Advanced SQL: Formulated complex analytical views, modern window function rankings (DENSE_RANK), and automated audit triggers.

(5) User Experience: Delivered an intuitive, responsive user interface with real-time capacity indicators, interactive schedules, and toast notifications, verified through an exhaustive test matrix achieving 100% pass rates.

12.3 Skills Learned

Throughout the engineering lifecycle of this project, the student team acquired comprehensive, practical competencies across multiple software domains: Advanced Relational Database Design: Schema normalization (1NF–BCNF), B-Tree indexing, execution plan analysis, trigger formulation, and ACID transaction handling in MySQL.

Backend & API Engineering: Asynchronous programming in Node.js, RESTful API contract design, promise-based connection pooling, JWT security implementation, and error middleware design in Express.

Frontend Development: Declarative component architecture in React 18, Single Page Application routing, context state synchronization, and modular CSS styling; Quality Assurance: Formulating systematic test suites, Postman API testing, and live database state verification.

12.4 Final Remarks

The Student Course Enrollment and Distributed Academic Management System establishes a solid, scalable digital foundation for modern academic institutions. It demonstrates that combining sound database engineering principles with clean full-stack architectural design yields software that is both highly dependable and exceptionally user-friendly.

The project stands as a testament to the practical application of Database Software Engineering and Distributed Backend Development principles, fulfilling all requirements for the Bachelor of Technology degree at Koneru Lakshmaiah Education Foundation.

CHAPTER 13 – REFERENCES

This chapter lists the authoritative academic literature, official technical documentation, standards specifications, and project source repositories consulted during the design and development of the system.

13.1 Project Source Materials

The primary implementation authority is the project source repository containing: client Single Page Application modules (src/), backend Express route controllers (server/routes/), database DDL and DML scripts (server/db/schema.sql, initDb.js, seed.js), configuration templates (server/.env.example), package manifests (package.json), and the official Postman collection (postman/Student_Portal_API.postman_collection.json).

All software behaviors, entity relationships, and architectural descriptions presented throughout this documentation report are directly substantiated by the verifiable source code artifacts.

13.2 Technical Documentation

Official framework and runtime documentation consulted includes: (1) React Documentation: <https://react.dev/> — Component lifecycles, hooks, and context APIs; (2) React Router DOM v6: <https://reactrouter.com/> — Declarative client-side routing; (3) Node.js v20+ LTS Documentation: <https://nodejs.org/docs/> — V8 engine and asynchronous event loop.

(4) Express.js Application Framework: <https://expressjs.com/> — Middleware architecture and REST routing; (5) Vite Next-Generation Tooling: <https://vite.dev/> — Module bundling and hot module replacement; (6) MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/> — InnoDB engine internals, transaction isolation, and stored routines.

13.3 Database and API References

Standard technical specifications and database design literature referenced: (1) Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson — Relational modeling, functional dependencies, and normalization theory.

(2) Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill — Transaction processing, concurrency control, and ACID properties; (3)

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Ph.D. dissertation) — RESTful API principles.

(4) OpenAPI 3.0 Specification: <https://spec.openapis.org/oas/v3.0.0> — RESTful interface documentation standards; (5) RFC 7519: JSON Web Token (JWT) — Stateless session security.

13.4 Citation and Verification Note

All cited literature, documentation manuals, and specifications were actively utilized during the design and validation phases. Code implementations were verified against official reference standards to ensure compatibility, security compliance, and performance optimization.

This report intentionally reflects the exact technologies implemented in the verified repository and avoids asserting third-party tools not represented in the codebase.

CHAPTER 14 – APPENDICES

This chapter compiles supplementary technical documentation, including full project directory trees, REST API specifications, SQL schema DDL listings, diagnostic queries, user operation manuals, technical glossaries, and entity cardinality summaries.

14.1 Project Directory Structure

The project repository follows a clean, modular directory structure separating frontend client code, backend server routes, database initialization routines, and test collections:

```

Student-Portal-Pro/ └── dist/                # Compiled production frontend assets └── docs/
# Architecture plans and syllabus specifications └── postman/                # Postman API test
collection JSON └── scripts/                # Automation scripts (build.mjs, setup.mjs) └──
server/                # Backend Node.js/Express application | └── config/db.js        #
MySQL connection pool configuration | └── db/                # Database DDL schema,
initDb.js, and seed data | └── middleware/                # Authentication and security filters |
└── routes/                # Decoupled REST API route controllers | └── services/        #
Business logic, validation, security rules | └── .env                # Active environment
secrets (local) | └── app.js                # Express application setup | └── server.js    #
Server entry point and port listener └── src/                # Frontend React 18 Single Page
Application | └── components/                # Reusable UI widgets (Header, Sidebar, Cards) |
└── context/AppContext.jsx # Global application state management | └── pages/
# Page-level route views (Dashboard, Courses, etc.) | └── pages/admin/        #
Administrative management views | └── services/api.js        # Centralized HTTP API client
| └── App.jsx                # Application layout shell | └── AppRoutes.jsx        # Route
tree and ProtectedRoute guards | └── main.jsx                # React root DOM entry point
└── package.json            # Project dependencies and npm automation scripts └── SETUP-
WINDOWS.bat                # 1-click Windows setup batch script └── START-WINDOWS.bat
# 1-click Windows server launch script

```

14.2 API Endpoint Reference

The API provides a fully RESTful interface adhering to OpenAPI 3.0 conventions. All sensitive routes require an Authorization: Bearer <token> header.

The reference table below outlines the endpoint inventory, access privileges, operation descriptions, and expected HTTP status codes:

HTTP Method	Endpoint Path	Protected?	Description	Success Code
POST	/api/auth/register	No	Self-service student account registration	201 Created
POST	/api/auth/login	No	Authenticates user and returns JWT token	200 OK
GET	/api/auth/me	Yes	Retrieves profile of authenticated user	200 OK
GET	/api/students/:id	Yes	Retrieves student profile with academic summary metrics	200 OK
PUT	/api/students/:id	Yes	Updates editable student contact fields	200 OK
GET	/api/health	No	Health check reporting server and DB status	200 OK
GET	/api/courses	No	Fetches complete active course catalog	200 OK
GET	/api/courses/:id	No	Fetches course details, syllabus, and outcomes	200 OK
GET	/api/enrollments/:id	Yes	Retrieves active enrollments for a student	200 OK
POST	/api/enrollments	Yes	Enrolls student in course; decrements seats	201 Created
DELETE	/api/enrollments	Yes	Drops course enrollment; increments seats	200 OK
GET	/api/grades/:id	Yes	Retrieves published examination grades and GPA	200 OK
GET	/api/timetable/:id	Yes	Retrieves student's weekly schedule matrix	200 OK
GET	/api/notifications/:id	Yes	Retrieves student notification alerts	200 OK
PATCH	/api/notifications/:id/read	Yes	Marks individual notification as read	200 OK
GET	/api/admin/overview	Yes (Admin)	Aggregates campus-wide telemetry metrics	200 OK
GET	/api/admin/students	Yes (Admin)	Retrieves complete student directory	200 OK

GET	/api/admin/faculty	Yes (Admin)	Retrieves faculty directory and assignments	200 OK
-----	--------------------	-------------	---	--------

14.3 Example SQL and Integrity Rules

The database schema enforces data integrity through declarative table constraints and foreign key relationships. The DDL excerpt below highlights key integrity definitions:

```
-- Declarative Domain Constraints in students table: ALTER TABLE students ADD
CONSTRAINT chk_gpa CHECK (current_gpa BETWEEN 0 AND 10); ALTER TABLE
students ADD CONSTRAINT chk_credits CHECK (total_credits > 0); -- Unique and Foreign
Key Constraints in enrollments table: ALTER TABLE enrollments ADD CONSTRAINT
unique_student_course UNIQUE (student_id, course_id); ALTER TABLE enrollments ADD
CONSTRAINT fk_enroll_student FOREIGN KEY (student_id) REFERENCES students(id)
ON DELETE CASCADE; ALTER TABLE enrollments ADD CONSTRAINT
fk_enroll_course FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE
RESTRICT; -- Course Capacity Invariants: ALTER TABLE courses ADD CONSTRAINT
chk_course_credits CHECK (credits BETWEEN 1 AND 6); ALTER TABLE courses ADD
CONSTRAINT chk_seat_capacity CHECK (total_seats > 0 AND available_seats >= 0 AND
available_seats <= total_seats);
```

14.4 Example SQL Queries

The following analytical queries demonstrate multi-table joins, aggregations, and window functions used to extract operational insights from student_portal_db:

```
-- 1. Real-time Course Occupancy Analysis: SELECT c.course_code, c.course_name, c.credits,
c.total_seats, c.available_seats,          ROUND((((c.total_seats - c.available_seats) * 100.0 /
c.total_seats), 1) AS occupancy_pct FROM courses c ORDER BY occupancy_pct DESC; --
2. Student Academic Standing with Aggregated Marks: SELECT s.roll_number, s.full_name,
s.department,                             COUNT(DISTINCT e.id) AS enrolled_courses,
ROUND(AVG(g.total_marks), 2) AS average_score,      s.current_gpa FROM students s
LEFT JOIN enrollments e ON e.student_id = s.id AND e.status = 'Enrolled' LEFT JOIN grades
g ON g.student_id = s.id GROUP BY s.id ORDER BY s.current_gpa DESC; --
3. Departmental Student Rankings using Window Function: SELECT roll_number, full_name,
department, current_gpa,    DENSE_RANK() OVER (PARTITION BY department ORDER
BY current_gpa DESC) AS dept_rank FROM students ORDER BY department, dept_rank;
```

14.5 Example API Requests

Developers and evaluators can interact directly with the REST API using curl or Postman. The examples below demonstrate core request payloads:

```
# 1. Authenticate Student Session: curl -X POST http://localhost:5000/api/auth/login \ -H
'Content-Type: application/json' \ -d
'{"studentIdOrEmail":"2520030477","password":"Student@2026!"}' # 2. Course Enrollment
Request (with JWT Bearer Token): curl -X POST http://localhost:5000/api/enrollments \ -H
'Content-Type: application/json' \ -H 'Authorization: Bearer <YOUR_JWT_TOKEN>' \ -d
'{"courseId":"DBMS","sectionId":"DBMS-SEC1"}' # 3. Retrieve Personalized Weekly
Timetable: curl -X GET http://localhost:5000/api/timetable/2520030477 \ -H 'Authorization:
Bearer <YOUR_JWT_TOKEN>'
```

14.6 User Guide – Student Workflow

Step-by-step operational guide for student portal interactions: (1) Access Portal: Open browser to <http://localhost:5000> and log in using registered student roll number and password.

(2) Dashboard: Review cumulative GPA, enrolled course summary, credit totals, and current attendance standing; (3) Select Academic Cluster: Navigate to 'My Cluster' to select an academic cohort track, unlocking designated course sections.

(4) Browse & Enroll: Open 'Courses', inspect course prerequisites and meeting times, select an open section, and click 'Enroll Now'; (5) Review Schedule: Navigate to 'Timetable' to view the generated weekly schedule; (6) View Grades: Access 'Grades' to inspect internal, midterm, and final examination marks.

14.7 User Guide – Administrator Workflow

Operational guide for university administrators: (1) Admin Login: Authenticate using administrator credentials (admin / Admin@2026!) to access privileged management consoles.

(2) Telemetry Inspection: Review total enrolled students, course catalog counts, active registrations, and section occupancy percentages on the Admin Dashboard.

(3) Student Records Management: Navigate to 'Student Directory' to search student profiles, inspect registration histories, and apply authorized CGPA overrides with required audit justifications.

(4) Audit Review: Open 'Audit History' to review immutable logs of student course enrollments, drops, and administrative modifications.

14.8 Glossary

Definition of core technical terms: ACID: Atomicity, Consistency, Isolation, Durability—foundational properties of database transactions. B-Tree Index: A balanced tree data structure utilized by MySQL InnoDB to accelerate search queries from $O(N)$ to $O(\log N)$.

Candidate Key: A minimal superkey capable of uniquely identifying tuples in a relation.

Foreign Key (FK): A referential constraint linking a column to a primary key in another table.

JWT: JSON Web Token—a compact, URL-safe means of representing signed claims between parties.

Normalization: The systematic formal process of decomposing relations to eliminate redundancy and update anomalies (1NF, 2NF, 3NF, BCNF). REST: Representational State Transfer—a stateless architectural style for distributed web services. SPA: Single Page Application—a web application that dynamically rewrites the current web page rather than loading entire new pages from a server.

14.9 Final Submission Checklist

Pre-submission verification checklist: [X] MySQL 8.0 server verified running on port 3306; [X] student_portal_db schema loaded with all tables, views, triggers, and procedures intact; [X] Environment configuration validated with secure JWT_SECRET in server/.env.

[X] Frontend production bundle compiled successfully in dist/; [X] Express backend server listening and responding on port 5000; [X] Complete student user journey executed successfully: Login -> Cluster Selection -> Course Enrollment -> Timetable -> Grades.

[X] Full test case matrix (TC-01 through TC-14) verified with 100% pass rates; [X] Formal documentation report compiled with all 14 chapters, institutional details, and academic certificates.

14.10 Database Object Inventory

The table below catalogs the principal database objects defined within the authoritative student_portal_db schema, including tables, views, triggers, and stored routines:

Object Type	Object Identifier	Description
Table	students	Core student identity, credentials, and academic summary metrics
Table	admins	Authorized administrative user accounts and roles
Table	courses	Institutional course catalog with credit and seating limits
Table	course_sections	Classroom offerings, weekly schedules, and instructor assignments
Table	enrollments	Associative registration table resolving M:N student-course relationships
Table	grades	Itemized assessment marks (internal, midterm, final) and grade points
Table	timetable	Student weekly schedule slots indexed by weekday and time
Table	notifications	Persistent student event alerts and message queues
Table	enrollment_audit_log	Immutable trigger-generated audit trail of registration events
Table	academic_clusters	Curriculum cohort clusters grouping course sections
Table	faculty	Teaching staff records: designation, department, contact details, and qualifications
Table	cluster_groups	Cohort groups belonging to an academic cluster
Table	section_clusters	Links course sections to cluster groups to enforce cohort boundaries
Table	student_cluster_choices	Records each student's chosen cluster per semester and whether the choice is locked
Table	academic_audit	Audit trail of administrative CGPA overrides with reason and timestamp

View	v_student_academic_summary	Aggregated student enrollments, credits, GPA, and attendance
View	v_course_enrollment_stats	Real-time course section capacity, filled seats, and occupancy percentage
View	v_student_rankings	Departmental and institutional ranks computed via DENSE_RANK() window function
Trigger	tr_enrollment_insert	AFTER INSERT trigger automatically logging new course enrollments
Trigger	tr_enrollment_update	AFTER UPDATE trigger logging course drops and section switches
Procedure	sp_student_transcript	Compiles and returns an official grade transcript for a student
Function	fn_calculate_student_avg	Calculates and returns the overall average marks obtained by a student

14.11 API Route Inventory

The table below enumerates the documented backend REST route handlers mounted on the Express application server:

Route Module	HTTP Method	Path	Controller Function
auth.routes.js	POST	/api/auth/register	Registers new student with hashed password
auth.routes.js	POST	/api/auth/login	Validates credentials and issues signed JWT
auth.routes.js	GET	/api/auth/me	Returns authenticated user profile
students.routes.js	GET	/api/students/:id	Returns student profile with academic view metrics
students.routes.js	PUT	/api/students/:id	Updates editable student contact fields
courses.routes.js	GET	/api/courses	Retrieves all active course offerings with seat counts
courses.routes.js	GET	/api/courses/:id	Retrieves course details, prerequisites, and syllabus
enrollments.routes.js	GET	/api/enrollments/:id	Retrieves active course enrollments for student
enrollments.routes.js	POST	/api/enrollments	Executes capacity-checked course enrollment
enrollments.routes.js	DELETE	/api/enrollments	Drops enrolled course and recovers seat capacity
grades.routes.js	GET	/api/grades/:id	Retrieves course assessment marks and CGPA
timetable.routes.js	GET	/api/timetable/:id	Generates weekly schedule matrix from enrollments
notifications.routes.js	GET	/api/notifications/:id	Retrieves student notification alerts

notifications.routes.js	PATCH	/api/notifications/:id/read	Marks an individual notification as read
admin.routes.js	GET	/api/admin/overview	Aggregates campus telemetry KPIs and occupancy
admin.routes.js	GET	/api/admin/students	Retrieves complete student administrative directory

14.12 Example Enrollment Data Flow

The transactional course enrollment logic executes atomically through the following pseudo-code, which the enrollment controller runs as a single database transaction:

```
START TRANSACTION; -- Step 1: Validate existence and retrieve current section capacity
SELECT available_seats, total_seats FROM courses WHERE id = ? FOR UPDATE; -- Step
2: Invariant Check - verify open seating capacity IF available_seats <= 0 THEN SIGNAL
SQLSTATE '45000' SET MESSAGE_TEXT = 'Course section is fully enrolled';
ROLLBACK; END IF; -- Step 3: Insert new enrollment record (unique key blocks
duplicates) INSERT INTO enrollments(student_id, course_id, section_id, status)
VALUES(?, ?, ?, 'Enrolled'); -- Step 4: Decrement course seating capacity UPDATE
courses SET available_seats = available_seats - 1 WHERE id = ?; -- Step 5: Trigger fires
automatically: -- tr_enrollment_insert records immutable entry into enrollment_audit_log
COMMIT;
```

14.13 Test Results Recording Sheet

The test recording sheet documents the empirical verification results achieved across all critical software test scenarios in the laboratory environment:

Test ID	Scenario Description	Expected Result	Actual Result	Status
TC-01	GET /api/health	HTTP 200, status: online	Matches Expected	PASS
TC-02	Valid Student Login	HTTP 200, returns user + JWT	Matches Expected	PASS
TC-03	Invalid Password Submission	HTTP 401 Unauthorized	Matches Expected	PASS
TC-04	Duplicate Roll Registration	HTTP 409 Conflict	Matches Expected	PASS
TC-05	Course Catalog Retrieval	HTTP 200, full course list	Matches Expected	PASS
TC-06	Nonexistent Course Lookup	HTTP 404 Not Found	Matches Expected	PASS
TC-07	Course Section Enrollment	HTTP 201 Created, seat decremented	Matches Expected	PASS
TC-08	Duplicate Enrollment Attempt	HTTP 409 Conflict	Matches Expected	PASS
TC-09	Zero-Capacity Enrollment	HTTP 400 Bad Request	Matches Expected	PASS
TC-10	Course Withdrawal / Drop	HTTP 200 OK, seat released	Matches Expected	PASS
TC-11	Weekly Timetable Retrieval	HTTP 200, sorted schedule slots	Matches Expected	PASS
TC-12	Grade Transcript Retrieval	HTTP 200, marks and CGPA accurate	Matches Expected	PASS
TC-13	Administrative Telemetry	HTTP 200, counts match MySQL DB	Matches Expected	PASS
TC-14	Audit Trigger Validation	Audit row created in DB log	Matches Expected	PASS

14.14 ER Relationship Summary

The relational schema topology establishes cardinality mappings that guarantee data integrity across academic entities:

- STUDENTS 1 ———< ENROLLMENTS >———— 1 COURSES (Many-to-Many resolved by associative Enrollments entity)
- STUDENTS 1 ———< GRADES >———— 1 COURSES (Evaluation records linking students to individual subjects)
- COURSES 1 ———< COURSE_SECTIONS (1:N relationship mapping courses to lecture sections and classrooms)
- FACULTY 1 ———< COURSE_SECTIONS (1:N relationship mapping instructors to assigned sections)
- STUDENTS 1 ———< TIMETABLE (1:N relationship mapping students to weekly schedule time slots)
- STUDENTS 1 ———< NOTIFICATIONS (1:N relationship providing individualized student alert queues)
- ENROLLMENTS 1 ———> ENROLLMENT_AUDIT_LOG (Automated trigger logging all registration transitions)
- ACADEMIC_CLUSTERS 1 ———< CLUSTER_GROUPS 1 ———< SECTION_CLUSTERS (Cohort grouping model)

These cardinality constraints ensure that no orphan records can exist within student_portal_db, and all academic relationships adhere strictly to third normal form.